

Part 2. Neural (LLM) scaling behaviors

Scaling Laws 的核心思想

先训练很多小模型 → 从中学
到规律 → 外推到大模型

1.Data vs performance

“Are there simple rules that determine how data affects performance?”

- ✓ 在一定区间内, 性能提升是“可预测的”
- ✓ 可以用于工程决策

2. Data vs model size

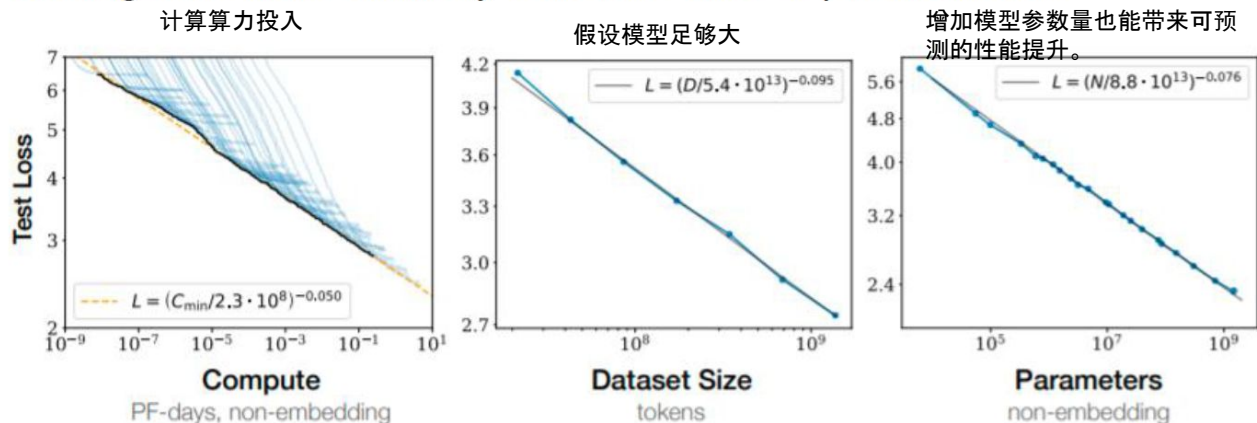
Do we train on more data or bigger models?

3. Hyper-parameters vs performance

“How should we set hyperparameters on the big model??”

Scaling laws – power law relationships for many factors

These scaling laws hold on *many* different kind of phenomena!



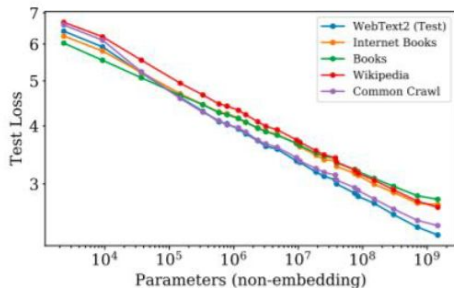
Scaling Laws 是非常确立 (well-established) 的规律。当你把 **模型性能 (Test Loss, 测试损失)** 作为 Y 轴, 把各种资源要素作为 X 轴, 并在 **双对数坐标系 (Log-Log Plot)** 下绘制时, 它们呈现出明显的 **线性关系**。性能与资源之间存在 **幂律 (Power Law)** 关系: 随着资源的指数级增加, 错误率会以可预测的速度线性下降。

1 PetaFLOPs 的速度连续运行 1 天所产生的总运算次数

当我们测试数据量的缩放时, 我们假设模型足够大 (Model size is much much bigger), 大到不会成为瓶颈, 这样才能单纯看到数据量增加带来的效果。

They even hold in non-standard settings (when train $\neq$ test)

这些定律不仅在标准的训练集上成立, 即使测试数据的分布与训练数据不同 (Train / Test) 在“非标准设置”下也成立。



Data vs performance

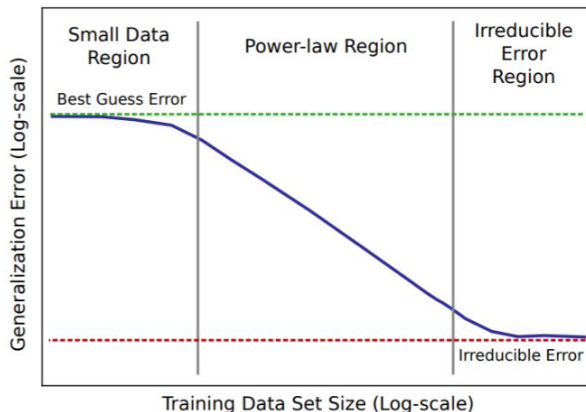
What's a data scaling law?

Data scaling laws : simple formula that maps dataset size (n) to error

What do we expect out of scaling laws?

Monotonic, logistic-like curves

1. **理论预期**：根据 VC 维和泛化界理论，随着数据量增加，我们预期误差下降的速度应该是 $1/\sqrt{N}$ (斜率约为 -0.5)。
2. **现实情况 (Scaling Laws)**：讲师展示的 PPT (如 Kaplan 或 Hestness 的图) 显示，在大语言模型中，误差下降的斜率 (幂律指数) 往往比理论预测的要慢得多 (例如斜率可能是 -0.095, 远小于 -0.5)。



- **小数据区域 (Small Data Region / Best Guess Error)**:
 - 在数据非常少的时候，模型基本上是在“瞎猜”。在这个阶段，增加少量数据可能不会带来明显的性能提升，模型处于随机猜测的水平 (图中绿色的虚线水平线)。
- **幂律区域 (Power-law Region)**:
 - 这是图中中间下降的那一段直线 (在对数坐标下)。
 - 一旦跨过了随机猜测的门槛，模型就进入了“可预测的缩放区域”。
 - 在这里，测试误差随着数据量的增加呈线性下降 (Power Law 关系)。这是我们最关心的区域，因为它可预测的。
- **不可约误差区域 (Irreducible Error Region)**:
 - 这是图中右下角趋于平缓的区域。
 - 随着数据量无限增加，模型性能最终会触及天花板。这个天花板是由数据本身的噪音或任务本身的难度决定的，无论怎么加数据也无法消除这部分误差 (图中红色的虚线水平线)。

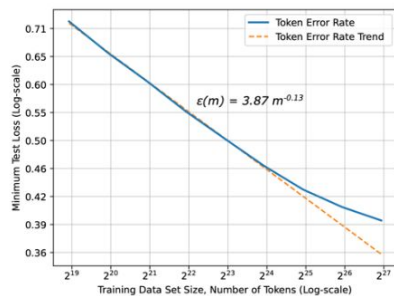
[Hestness+ 2017]

Scaling law exponents: an intriguing mystery

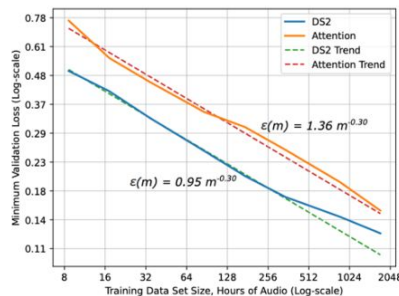
Fact: Similar arguments show most ‘classical’ models (regression, etc) have $\frac{1}{n}$ scaling

This means we should see $y = -x + C$

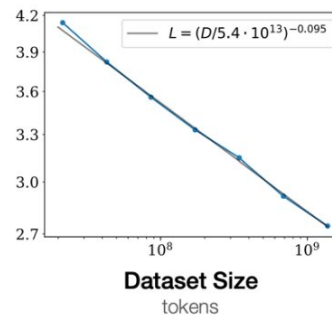
What do we find in neural scaling laws?



Machine translation



Speech



Language modeling

Very different from predictions.. Why might this be?

Detour: scaling laws for (nonparametric) learning

Neural nets can approximate arbitrary functions. Lets turn that into an example.

Input: $x_1 \dots x_n$ uniform in 2D unit box. $y_i = f(x_i) + N(0,1)$

Task: estimate $f(x)$

Approach: cut up the 2D space into boxes with length $n^{-\frac{1}{4}}$

What's our estimation error?

Informally, we have $\sqrt{n}$ boxes, each box gets $\sqrt{n}$ samples.

$$Error \approx \frac{1}{\sqrt{n}} + (\text{other smoothness terms})$$

In d -dimensions, this becomes $Error = n^{-1/d}$ - **This means scaling is $y = -\frac{1}{d}x + C$**

Takeaway: flexible 'nonparametric' learning has dimension dependent scaling laws.

现实中的语言模型斜率为什么这么小？

原因 1: 神经网络不是在估计一个参数

原文要点翻译：

神经网络并不是在做像“估计均值”那样的事情。
它们也不是在拟合一个线性模型。

相反，它们是在拟合一个**高度灵活的函数类**，也就是所谓的**非参数模型**（non-parametric models）。

原因 2: 非参数模型 → 维度决定收敛速度

假设我们在一个二维空间中采样数据，想要学习一个平滑的回归函数。

一个最简单的非参数方法是：

- “把空间切成很多小格子”
- “每个格子里取平均”

如果你这样做，你会发现误差随样本量下降的速度，与**空间维度直接相关**。

推广到 d 维：

$$\text{error} \sim n^{-1/d}$$

在非参数估计（如切格子）中，误差主要取决于“采样点的密度”。在低维空间增加一点点数据就能显著增加密度，但在高维空间，增加同样数量的数据，密度几乎没有变化。

如果你的函数类是高度灵活的（非参数），那么**你必然**会看到随数据量增长，误差下降得非常慢。

“套回”语言模型：

神经网络模型本质上是在拟合一个极其复杂的函数。

这个函数的输入是：

- 高维 token 序列
- 上下文相关
- 强依赖语义、语法、世界知识

因此，从统计角度来看，语言建模是一个**极高内在维度 (intrinsic dimension)**的问题。

所以就会得到：

如果问题的内在维度非常高，
那么幂律指数就会非常小。

这正是你看到的：

- 0.13
- 0.095
而不是 0.5 或 1.0

看到这么慢的 scaling 速度，并不意味着模型“学得不好”。

相反，这是一个**高度灵活、表达能力极强**的模型类所必然付出的代价。

Intrinsic dimensionality theory of data scaling laws

Some have made the following argument (Bahri 2021)

1. Scaling laws arise due to polynomial rates of learning $\frac{1}{n^\alpha}$
2. Some argue the slope α is closely connected to the *intrinsic dimensionality* of the data.

数据的内在维度 (d) 越高
 $\rightarrow$ Y轴数值越高
 $\rightarrow$ 代表斜率 α 越小
 $\rightarrow$ 误差下降越慢。

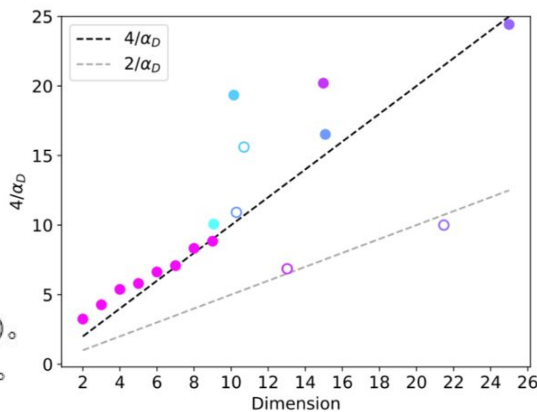
中的点:

粉色点 (Teacher-Student): 这是人造数据 (Synthetic Data)。

你知道它的维度是多少。你可以看到这些点完美地落在虚线上。

维度越高, α 确实越小 (Y轴值越大, α 越小)。

其他颜色的点 (CIFAR, MNIST等): 这是真实的图像数据。虽然不完美, 但总体趋势也是向右上倾斜。这意味着真实数据的内在维度斜率确实越平缓 (学习越慢)。



我们怎么知道误差下降慢 (α 小) 真的是因为维度 (d) 高造成的?

1. **Scaling Laws (缩放定律)** 的形式是多项式衰减: $\text{Error} \sim \frac{1}{n^\alpha}$.
 - 这里的 α 就是斜率 (学习速度)。 α 越大, 误差下降越快。
2. **核心结论:** 这个斜率 α 与数据的内在维度 (Intrinsic Dimensional)
 - 这正是我们刚才推导的结论: $\alpha \approx \frac{1}{d}$ (或者在这个图中是 $\frac{4}{d}$)。

intrinsic dimension are sketchy, and this is not airtight..

Other data scaling laws

Data scaling thus far: how does dataset size relate to performance?

Related question: how does dataset *composition* affect performance

数据**构成** (Composition) 和**质量**如何影响性能。

❖ Picking optimal data mixture using small scale models

数据的构成(即不同 类型数据的混合比例)主要影响 扩展曲线的**截距(offset)**, 而不是**斜率(slope)**;
可以利用小模型来高效地通 过回归等技术找出最佳的数据混合策略

❖ Deciding whether to repeat data or not

我们能否重复使用现有的数据进行多轮训练(Multi-epoch training)随着重复次数的增加, 数据的有效样本量(effective sample size)和独特 token 的价值会逐渐降低 ;大约在重复 4 个 epoch 之后, 收益就会迅速下降

❖ Combing the two and balancing quality with repetition rate

面临两个选择:

1. **重复高质量数据:** 比如把高质量的维基百科或书籍数据重复 训练 10 次 。
2. **引入新数据:** 引入质量较低但全新的数据

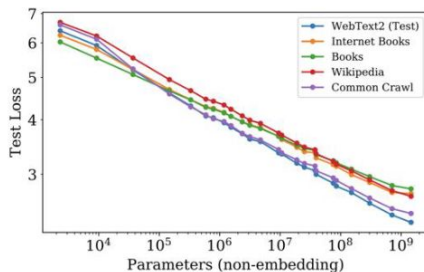
可以利用扩展定律来计算这个权衡点, 从而决定是 应该忍受重复数据的效用降低来 换取高质量, 还是应该忍受低质量数据来 换取新鲜度 。通过拟合扩展定律, 你可以预测在规模化训练时, 哪种策略(重复旧数据 vs. 引入新劣质数据)能带来更低的 Loss 。

Other advanced data scaling law: distribution shift

Data scaling thus far: how does dataset size relate to performance?

Related question: how does dataset *composition* affect performance

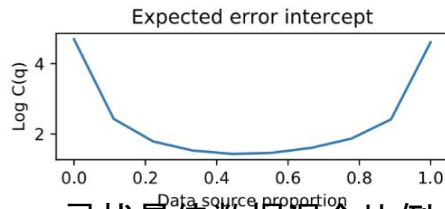
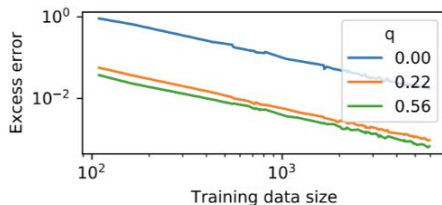
A: Data composition affects the offset, not the slope.



斜率 (Slope) 不变: 这意味着无论你用哪种数据, 随着模型变大, 性能提升的 **速度** (也就是 Scaling 的效率) 是基本一致的。
截距 (Offset) 改变: 数据的质量或类型决定了曲线的 **上下位置**。更好的数据 (如清洗过的 WebText2) 会让曲线整体向下平移 (Loss 更低), 而较差的数据会让曲线整体在上方。

[Kaplan+ 2021] 因为线是平行的, 不会交叉。你不需要训练一个巨型模型来验证哪个数据集更好

These ‘distribution shift’ scaling laws can tell us about the importance of collecting diverse data!



这个 U 型曲线的最低点, 就代表了 **最优的数据混合比例**

寻找最佳数据混合比例

[Hashimoto 2021]

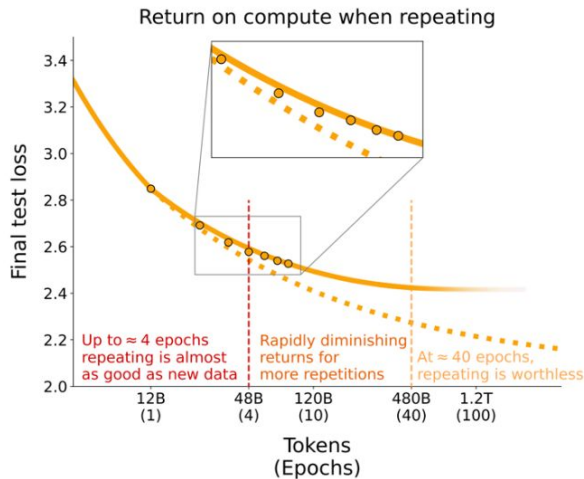
Scaling laws under data repetition

In practice, we have finite data – how does repeating examples affect scaling?

重复数据的收益递减

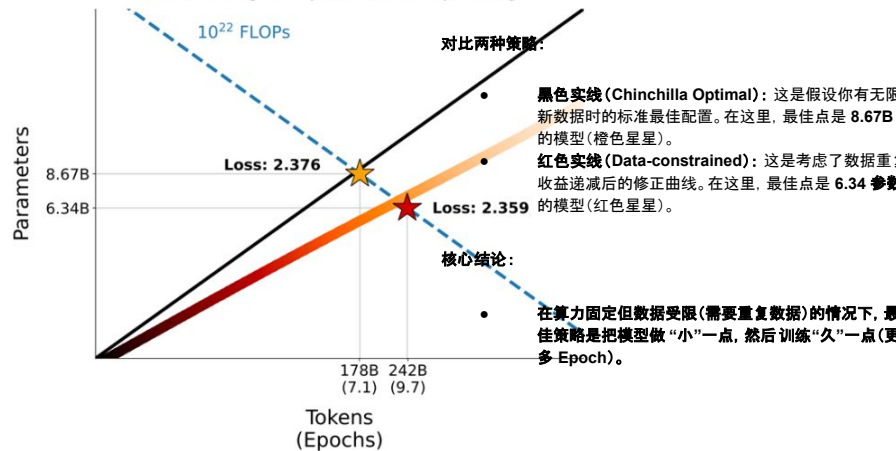
理想 vs. 现实:

- 虚线(橙色点状): 这是假设“重复的旧数据”和“全新的数据”价值一样时的理想曲线。如果真是这样, Loss 会一直顺滑下降。
- 实线(橙色实线): 这是实际观测到的曲线。你可以看到它逐渐变平, 偏离了理想路线。



计算资源的重新分配

Allocating compute when repeating



如果我算力 预算固定(比如 蓝色虚线所示的 10^{22} FLOPs), 但在数据有限 必须重复的情况下, 我该造多大的模型?

黑色实线 (Chinchilla Optimal): 这是假设你有无限新数据时的标准最佳配置。在这里, 最佳点是 8.67B 的模型 (橙色星星)。

红色实线 (Data-constrained): 这是考虑了数据重复收益递减后的修正曲线。在这里, 最佳点是 6.34B 参数的模型 (红色星星)。

在算力固定但数据受限(需要重复数据)的情况下, 最佳策略是把模型做“小”一点, 然后训练“久”一点(更多 Epoch)。

随着重复次数 R_D 增加, 指数项使得增加的收益迅速饱和。无论你重复多少次, 你能获得的“有效信息量”是有上限的。

所以, 面对数据荒 (Data Scarcity), 正确的工程决策不是盲目堆参数, 而是:

- 接受由于数据不足必须进行多轮训练 (Multi-epoch training) 的现实。
- 为了适应这种现实, 适当**缩减模型规模**(比如从 8.7B 缩减到 6.3B)。
- 利用省下来的算力, 让这个小型模型在有限的数据上跑更多的轮次(比如从 7.1 轮增加到 9.7 轮)。

有效数据 (Effective Data)

$$D' = U_D + U_D R_D^* (1 - e^{-\frac{R_D}{R_D^*}})$$

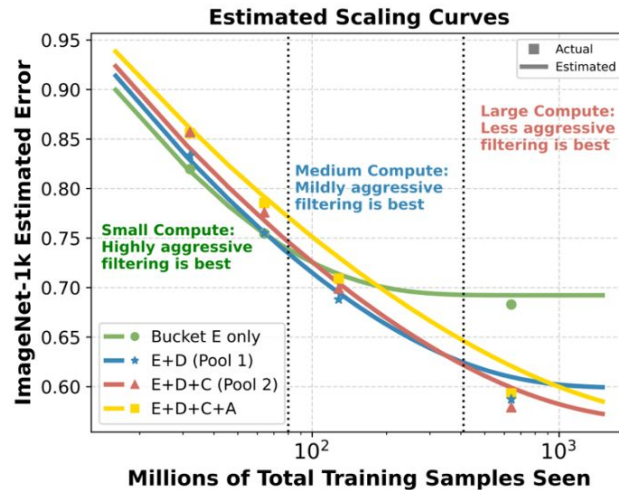
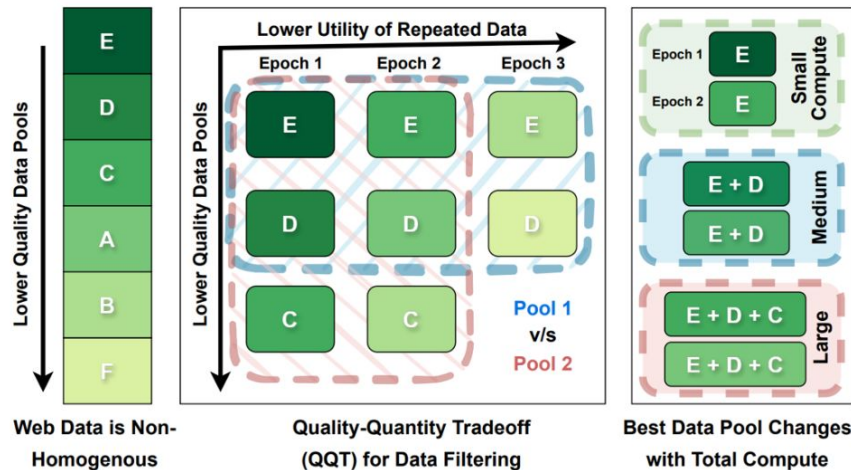
D' = Effective data
 U_D = Unique tokens
 R_D^* = Constant
 R_D = Repetition

Data selection scaling and accounting for finiteness

Given that repeated data is less valuable..

Data selection should then be adaptive to scale !

- 左侧图示 (Data Pools): 互联网上的数据质量参差不齐。
 - Bucket E: 最高质量的数据 (例如教科书、维基百科)。
 - Bucket F: 低质量的数据 (例如垃圾网页)。
- 决策困境:
 - 如果你只用最好的 Bucket E, 数据量很少。在大规模训练时, 你不得不疯狂重复这些数据。上一张幻灯片告诉我们, 重复会导致收益迅速递减。
 - 如果你引入 Bucket D, C, A, 数据量变大了, 可以避免重复。但是, 数据的平均质量下降了, 这本身也会损害模型性能。



如果你在训练一个小模型(或者预算有限), 你应该激进地过滤数据, 只保留最高质量的部分。如果你在训练一个超大模型(预算充足), 你应该放宽过滤标准, 接受一些低质量的数据, 以换取数据的新鲜度和规模, 避免“重复数据”带来的性能瓶颈。

Hyperparameter questions

We'll consider some of these choices in the context of the classic Kaplan scaling paper

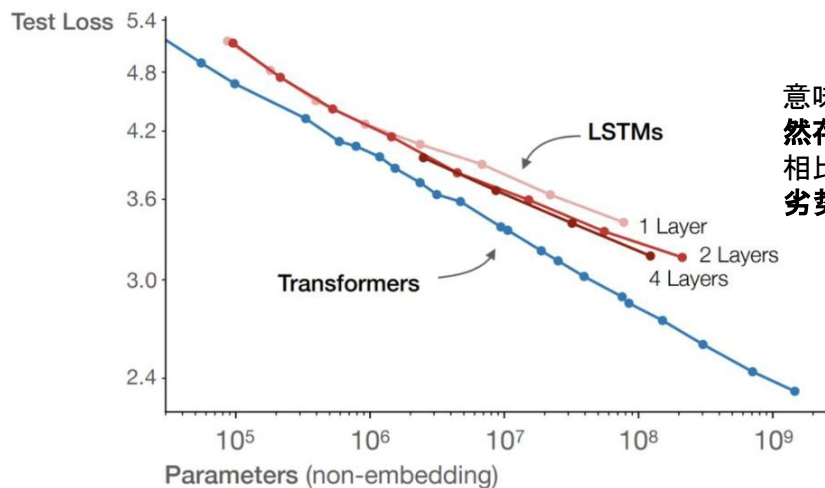
- Architecture
- Optimizer
- Aspect ratio / depth
- Batch size

1. Architecture: transformers vs LSTMs

Q: Are transformers better than LSTMs?

Brute force way: spend tens of millions to train a LSTM GPT-3

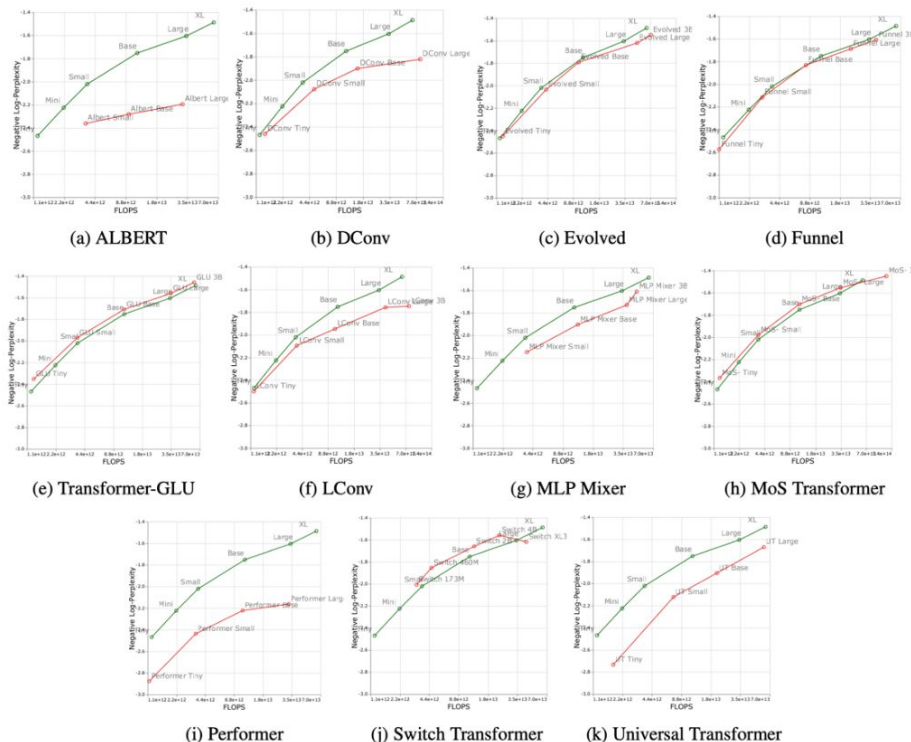
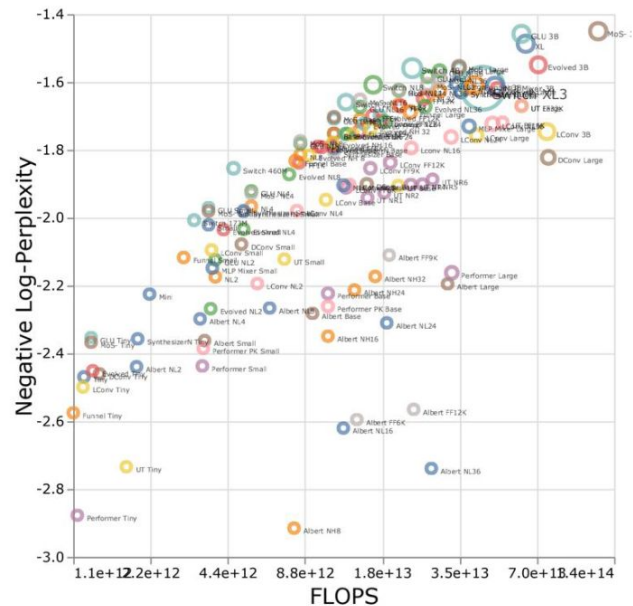
Scaling law way:



意味着随着模型变大, **Transformer 的优势依然存在, 不会被追平**; 这张图揭示了 LSTM 相比 Transformer 有一个无法逾越的 **常数级劣势**。

[Kaplan+ 2021]

1. Many architectures



但当我们把它们放大(Scale up)时,它们真的还能打败标准 Transformer 吗?

横轴: 训练计算量 (FLOPS)。

纵轴: 负对数困惑度 (Negative Log-Perplexity)。注意: 值越高越好 (因为困惑度越低越好, 取负号后就是越高越好)。

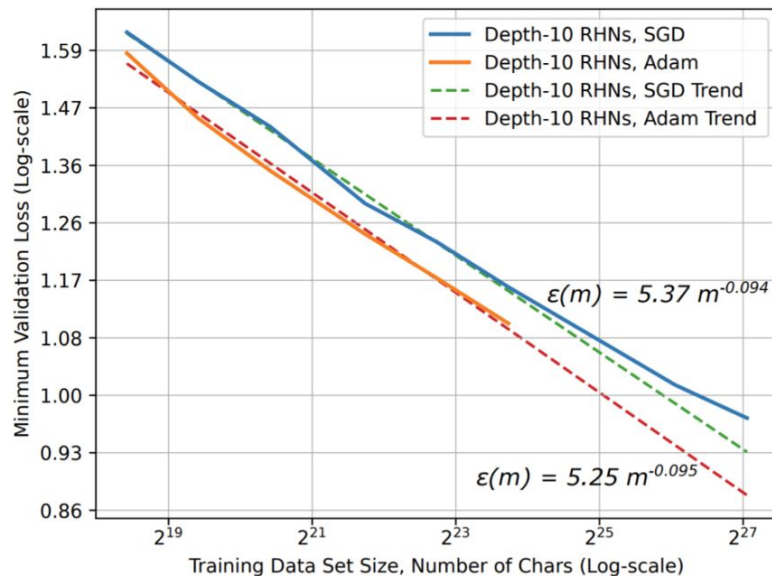
图 (e) Transformer-GLU: 使用了 Gated Linear Units。你可以看到红线略微优于绿线。这就是为什么现在的 Llama、Mistral 等模型都使用 SwiGLU 激活函数的原因。

图 (j) Switch Transformer: 这代表了 MoE (Mixture of Experts) 架构。它在同等算力下明显优于基线。这也解释了为什么现在像 Mixtral 和 GPT-4 这样的模型都转向了 MoE 架构。

Cross-architecture scaling in Tay et al.

2. Optimizer choice

What about ADAM vs SGD?



Adam 始终比 SGD 更有效率，而且这种优势是跨越规模的。无论数据量多少，Adam 都能在更少的数据下达到同样的 Loss，或者在同样的数据下达到更低的 Loss。

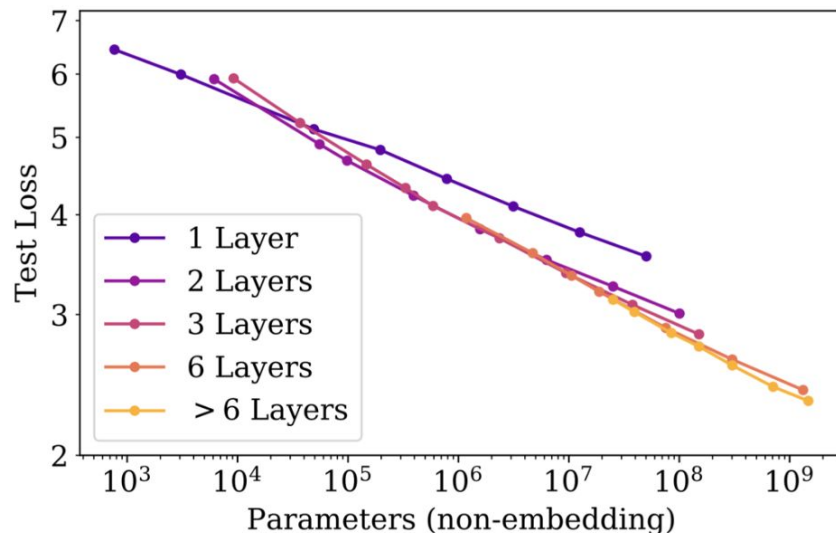
[Hestness+ 2017]

(Note, this is in 2017, so pre-transformers. RHN is recurrent highway nets)

3. Depth/Width: Number of layers

Does depth or width make a huge difference?

深度 (Depth, 层数) 和
宽度 (Width, 每层神经
元数量) 对模型性能有
多大影响?



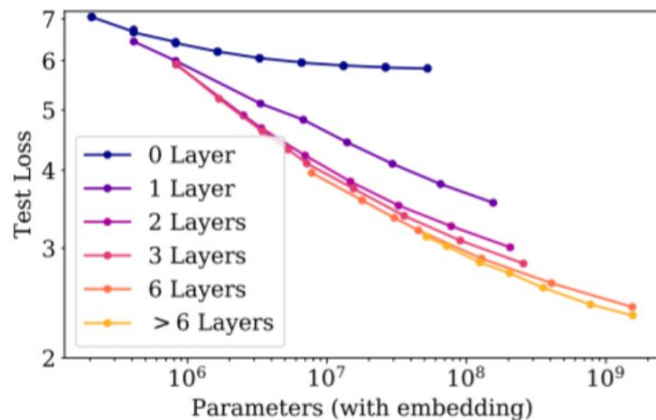
只要你的模型不是极端的
“浅”，具体的形状其实没那么
重要。

模型性能主要取决于 **总参数量**，而
不是具体的长宽比 (Aspect Ratio)

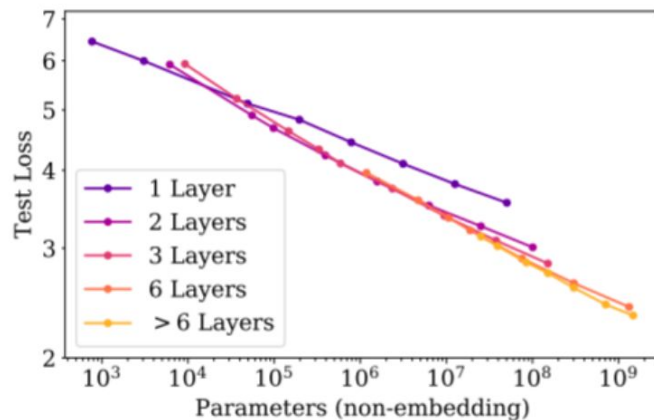
- 1 vs 2 layers makes a huge difference.
- More layers have diminishing returns below 10^7 params

3. Depth/Width: But not all parameters are made equal

We've been thinking about 'parameters' but not all parameters are equal



对于小模型或浅层模型，嵌入层(词表映射)占据了参数的绝大部分。但嵌入层只是一个查表操作，并不贡献复杂的逻辑推理能力



只有计算那些负责处理信息的参数(如 Attention 和 MLP 层的权重)，Scaling Law 才是准确的。

Embedding layer parameters don't behave the same!

如果你把“所有参数”都混为一谈，你的预测曲线就会变歪；但如果你排除掉 **嵌入层(Embedding Layer)** 的参数，曲线就会变得非常完美。

Related: recent papers on scaling laws for mixtures of experts.

在 MoE 中，模型的总参数量可能极其巨大(比如万亿级)，但在每次推理时只激活一小部分参数(稀疏激活)。如果你用“总参数量”去画 Scaling Law，曲线会完全失效。你必须计算“等效稠密参数(Equivalent dense parameters)”或“激活参数量”，才能看到真实的 Scaling 规律。

3. Depth/Width: and other Transformer hyperrs

Transformer 的其他超参数(如 长宽比、前馈层维度等)是否依赖于模型规模。

Do hyperparameters like the aspect ratio depend on scale?

超参数的最佳值是“跨规模稳定”的

无论你是训练几千万参数的小模型, 还是十几亿参数的大模型, 最佳的长宽比范围是不变的

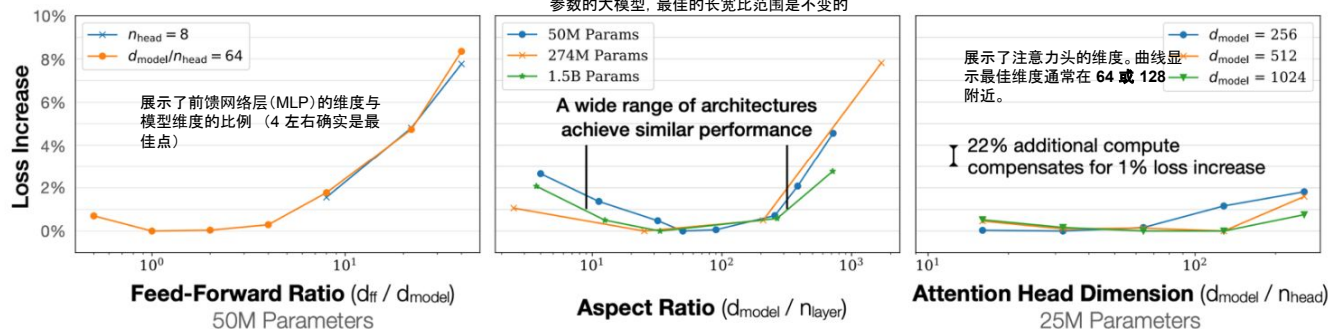


Figure 5 Performance depends very mildly on model shape when the total number of non-embedding parameters N is held fixed. The loss varies only a few percent over a wide range of shapes. Small differences in parameter counts are compensated for by using the fit to $L(N)$ as a baseline. Aspect ratio in particular can vary by a factor of 40 while only slightly impacting performance; an $(n_{\text{layer}}, d_{\text{model}}) = (6, 4288)$ reaches a loss within 3% of the $(48, 1600)$ model used in [RWC⁺19].

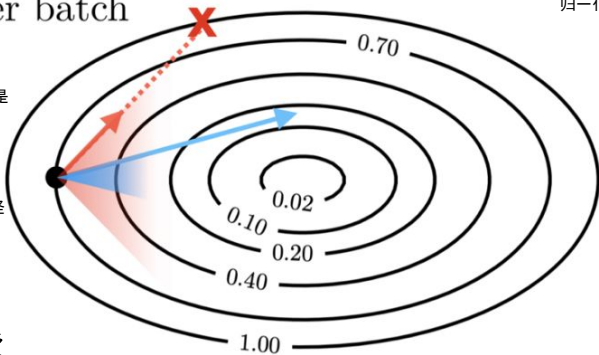
通用的超参数比例(如 MLP 比例为 4)在任何规模下都是接近最优的。

4. Batch size: Critical batch size

为了加快训练速度，我们可以无限制地增加 Batch Size (即同时并行处理更多数据) 吗？

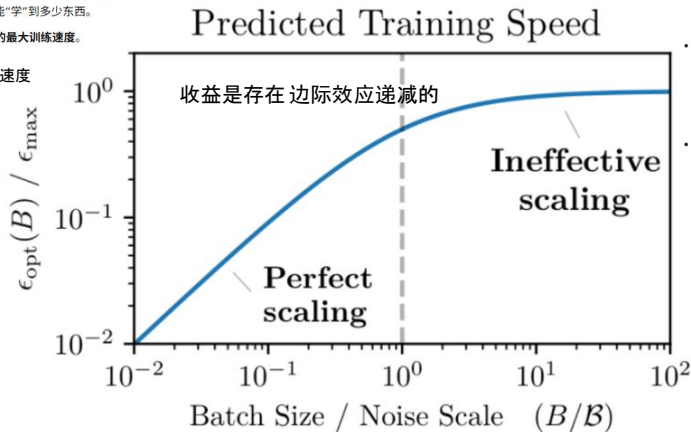
- Smaller batch
- Larger batch

- 背景：图中的同心圆代表 Loss Landscape (损失函数的等高线)，中心是最低点。
- 红色路径 (Smaller batch)：小 Batch Size 意味着梯度估计充满了噪声 (Noise)。为了不走偏，你必须使用较小的学习率，小心翼翼地走很多小步，路径看起来比较曲折。
- 蓝色箭头 (Larger batch)：大 Batch Size 的梯度非常精准 (去噪了)。你可以放心地使用很大的学习率，迈出一大步直达圆心。
- 权衡：增加 Batch Size 的本质是用更多的计算量 (并行处理更多数据) 来换取更少的步数 (更精准的梯度)。



ϵ (Epsilon): 在这里代表 训练速度 (Training Speed).
能下降多少，或者说模型能“学”到多少东西。
 ϵ_{max} (分母): 代表 理论上的最大训练速度。

归一化的训练速度



- 在 Perfect Scaling 区域 (小 Batch):
 - 主要矛盾是噪声。
 - 此时梯度很“吵”，增加 Batch Size 可以显著降低噪声。
 - 每多一点点数据，梯度就准一点，我们就能更快地接近最优解。
- 在 Ineffective Scaling 区域 (大 Batch):
 - 主要矛盾变成了偏差 (曲率)。
 - 此时 Batch Size 已经非常大了，梯度里的噪声已经可以忽略了。
 - 这时候你再增加数据，梯度也不会变得更准。
 - 但是！即使梯度完美，你也不能无限增大 (有 Curvature)，你沿直线走太远就会撞到“墙”。
 - 这种“墙”是由于地形弯曲而导致完美梯度也无法主导。

1. 两个对手：噪声 (Noise) vs. 偏差 (Bias/Curvature)

在训练神经网络时，阻碍我们一步步走到最优解的因素主要有两个：

- 噪声 (Noise/Variance): 因为我们只看了一小部分数据 (Batch)，所以算出来的梯度是“抖动”的，不完全代表真实方向。
- 偏差 (Bias/Curvature): 即使我们看完了所有数据，算出了完美的梯度，但梯度只是一个线性的指向。而真实的损失函数地形是弯曲的 (非线性的)。用直线的梯度去拟合曲线的地形，当你步子迈太大时，就会产生误差，这个由地形曲率带来的确定性误差，在这里被称为“Bias”。

Batch size – known to have strong diminishing returns past a certain point.

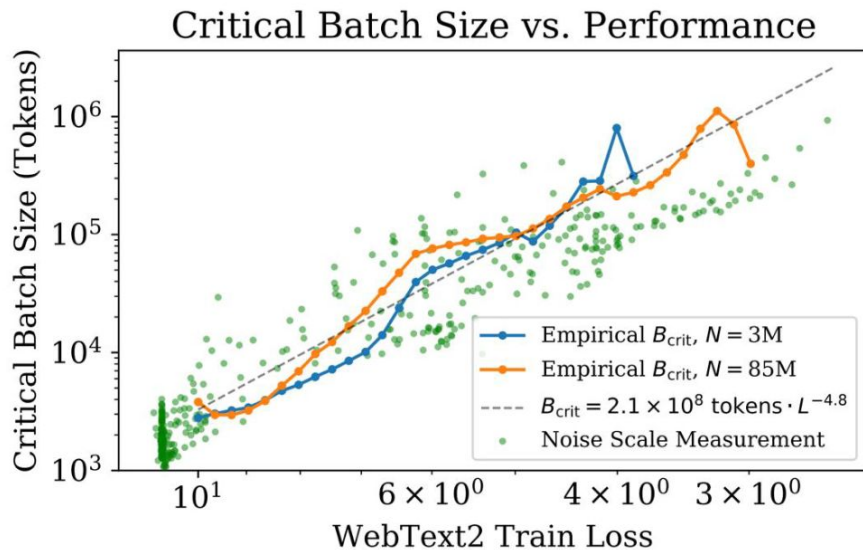
$$\text{Critical Batch Size} = \frac{\text{达到目标 Loss 所需的最少样本总量 (Min Examples)}}{\text{达到目标 Loss 所需的最少训练步数 (Min Steps)}}$$

Critical batch = min number of examples for target loss / min number of steps for target loss

之所以这个除法 ($\epsilon_{\min} / S_{\min}$) 的结果正好对应图中的转折点：这个点是“数据效率极限”和“时间效率极限”两条渐近线的交点。

4. Batch size: critical batch size

随着训练的进行 (Loss 变小), 临界 Batch Size 会变大; 像 Llama 3 这样的模型在训练后期会增加 Batch Size——因为当模型越来越接近最优解时, 由于梯度变小, 需要更多的数据来把梯度里的噪声压下去, 从而允许更大的 Batch Size 发挥作用



请注意坐标轴的方向。最左边是 10, 最右边是 3

这个发现对训练大模型 (如 Llama 3) 有巨大的指导意义:

- **不要一开始就上大 Batch:** 在训练刚开始时, Critical Batch Size 很小。如果你一开始就设一个很大的 Batch Size (比如 4M tokens), 你就是在浪费算力, 因为你超过了临界值, 进入了“无效扩展区”。
- **循序渐进 (Ramp-up):** 正确的做法是随着训练的进行, 逐步增加 Batch Size。这不仅能节省早期的显存和算力, 还能始终让训练保持在“高效扩展区”和“临界区”的边缘, 实现性价比最大化。
- 刚起步时 (高 Loss) 档位要低 (小 Batch), 速度起来后 (低 Loss) 要挂高档 (大 Batch)。

The smaller the loss target,
The bigger the batch

$$C_{\min}(C) \equiv \frac{C}{1 + B/B_{crit}(L)} \quad (\text{minimum compute, at } B \ll B_{crit})$$

这个公式用来计算计算效率。

- 它告诉我们在特定的 Loss 水平 L 下, 如果你使用的 Batch Size B 远小于临界值 $B_{crit}(L)$, 那么你的计算效率接近最优 ($C_{\min} \approx C$)。
- 一旦 B 超过了 B_{crit} , 分母变大, 有效算力 $C_{\min}$ 就会大打折扣。

4. Batch size: selecting the optimal batch

Q: as we increase both compute and model size, how should we scale training?

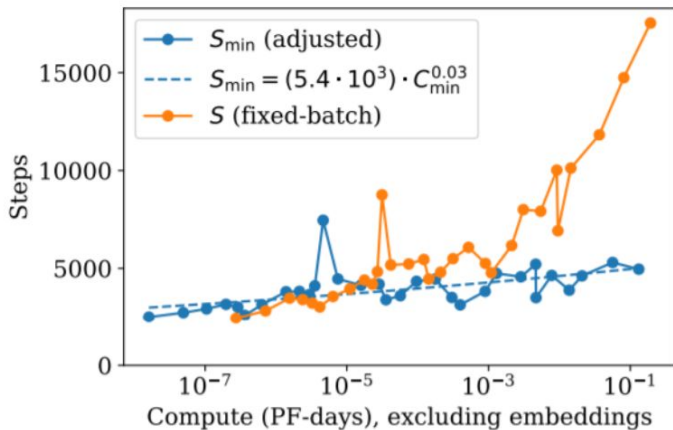
- Huge batches, same number of steps
- Fixed batches, more steps

当我们有更多的算力 (Compute) 用来训练更大的模型时, 我们应该如何分配这些算力? 是应该跑更多的步数 (Steps), 还是应该把 Batch Size 搞得更大?

Fixed batches (橙色策略): 保持 Batch Size 不变, 跑更多的步数 (例如训练时间从 1 个月变成 100 个月)。

Huge batches (蓝色策略): 保持步数不变, 把 Batch Size 变大 (例如用 100 倍的 GPU 并行处理数据, 训练时间依然是 1 个月)。

如果你死板地固定 Batch Size, 随着算力增加, 你不得不跑越来越多的步数 (线性增长)。这意味着训练时间会无限拉长。



蓝色线 (Adjusted / Optimal):

- 这是根据 Scaling Law 计算出的最优策略。
- **关键发现:** 这条线几乎是平的! 公式中的指数是 0.03 ($S_{\min} \propto C_{\min}^{0.03}$), 这意味着步数对算力极其不敏感。(基于临界批次大小 (Critical Batch Size) 与目标损失 (Target Loss) 之间的幂律关系推导出来的)

无论你是训练一个小模型, 还是训练一个超级巨大的模型, **最优的训练步数 (Steps) 几乎维持在同一个数量级。**

直观解释: 随着模型变得越来越聪明 (Loss 变低), 它对梯度的“分辨率”要求越来越高, 因此它“极其渴望”更多的数据来消除噪声。这种“对大 Batch 的渴望”增长得太快了, 以至于把你增加的所有算力 预算都“吃”掉了, 导致你根本不需要 (也没有 预算) 去跑更多的步数。

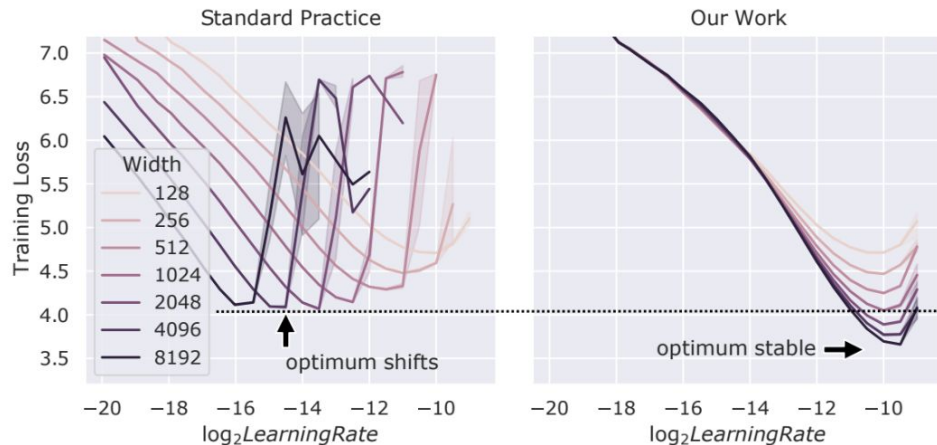
Good news for data parallel processing (?)

并行化的胜利: 既然最优步数 (Steps) 几乎不变, 这意味着当我们需要扩展训练规模时, 我们不需要等待更长的时间 (时间主要由步数决定)。

堆 GPU 就行: 我们只需要增加 Batch Size。而增加 Batch Size 是最容易通过“数据并行 (Data Parallelism)”来实现的——只需要增加更多的 GPU, 让它们同时处理更多的数据即可。

muP (Maximal Update Parametrization) 是一种旨在解决 大模型超参数 调优难题的模型参数化技术。它的核心作用是：让你在一个可以在几分钟内跑完的小模型上 调出的最佳学习率 (Learning Rate)，能够直接应用到训练需要几个月的大模型上，而无需重新搜索。

5. Learning rates: muP and scale-aware LR choices



方法 (Standard Parametrization, SP, 即 PyTorch 或 TensorFlow 模型宽度 (Width) 与最佳学习率之间存在复杂的依赖关系。

象：如幻灯片左图所示，随着模型宽度增加 (从 128 增加到 8192)，失函数的最低点 (最佳学习率) 会不断向左漂移。

果：这意味着大模型的最佳学习率通常比小模型要小很多。如果你一个小模型上找到了最佳 LR，这个数值在大模型上会导致训练发 (爆炸)。你必须通过昂贵的猜测或试错来寻找大模型的参数。

muP 是如何工作的？(核心机制)

muP 通过改变权重的初始化方式和 学习率缩放规则，使得在模型宽度趋于无限大时，神经网络每一层的激活值 (Activation) 和梯度更新量 (Update) 保持在合理的范围内。

Yang et al 2022

Table 2. μP function for a model M' that is r times the widths of M . If a parameter tensor has 2 dimensions that goes infinite when the model width goes infinite, it is “matrix-like” (e.g., a fully-connected hidden layer); if the number is 1 or 0, it belongs to the “others” class. Note that embedding layers are “others”. “Output” means the layer that maps an infinite dimension to a finite dimension, which is the word decoding layer (*lm_head*) in Transformers. A multiplier is a constant multiplied by a parameter tensor, which has a similar function to softmax temperature.

Hyperparameter (weight)	M	$M' \sim r$
AdamW learning rate (matrix-like)	l	l/r
AdamW learning rate (others)	l	l
Initialization variance (matrix-like)	σ	σ/r
Initialization variance (others)	σ	σ
Multiplier (output)	τ	τ/r
Multiplier (others)	τ	τ

- 矩阵类参数 (Matrix-like): 例如全连接层的权重矩阵 (W)。
 - 当宽度增加 r 倍时，这类参数的学习率通常需要除以 r ($LR \rightarrow LR/r$)。
 - 初始化的方差也可能需要相应的缩放 ($\sigma \rightarrow \sigma/r$)。
- 其他参数 (Others): 例如偏置 (Bias)、层归一化 (LayerNorm)、嵌入层 (Embedding)。
 - 这类参数的配置通常保持不变，或者遵循不同的缩放规律。

If we naively scale up – optimal learning rate depends on scale.

We need *scaling aware* initialization and learning rate scaling

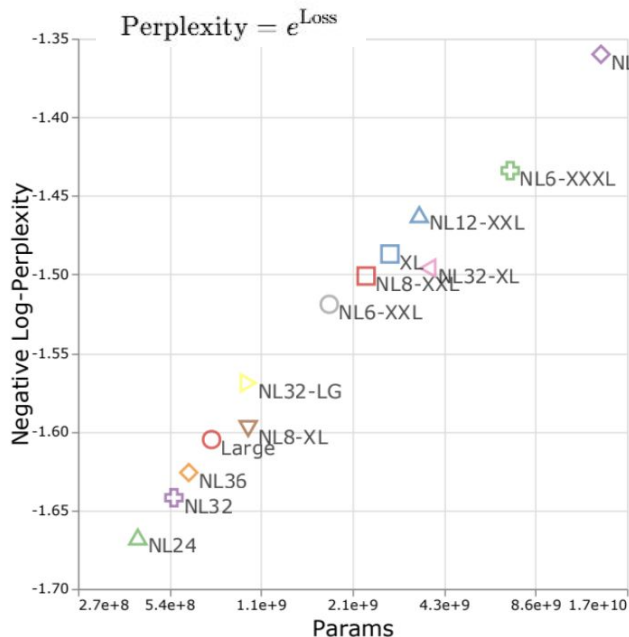
结果与优势: 曲线对齐 + 零样本迁移

Caution – scaling behaviors can differ downstream

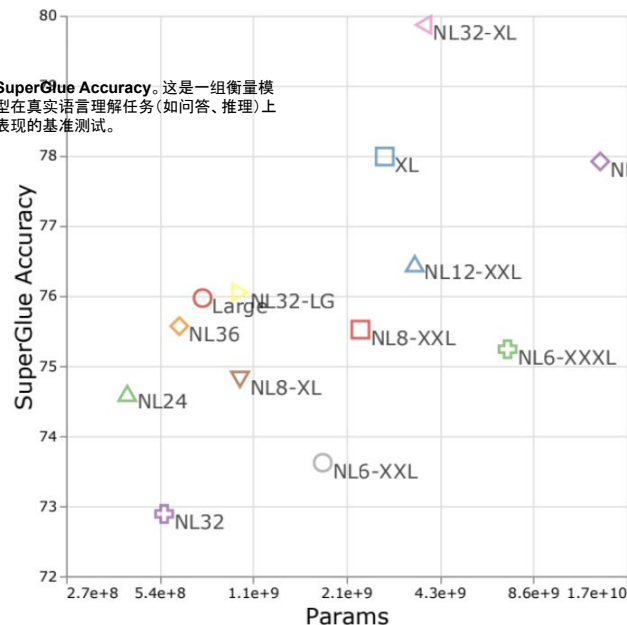
Thus far: scaling is predictable and depends mainly on parameters

Catch: downstream scaling can often be much less predictable

困惑度 (Perplexity, 简称 PPL): 当模型预测下一个 token 时, 它有多“确定”; $\log \text{Perplexity} = \text{loss}$



SuperGlue Accuracy. 这是一组衡量模型在真实语言理解任务(如问答、推理)上表现的基准测试。



结论: 在预训练阶段, **Scaling Law 非常有效**。只要你增加数据或算力, Perplexity 就会以极高的一致性下降(性能提升)。这个阶段, 具体的架构细节似乎不那么重要, 只有规模 (Scale) 才是王道。

Scaling Law 在下游任务上失效了(或者变得极其不可预测)。即使你的模型 Perplexity 更低, 也不保证它在做具体任务(如 QA、逻辑推理)时一定更聪明。

为什么会这样?

下游任务(Benchmarks)往往是非连续的、复杂的, 并且可能受限于特定的能力瓶颈(比如是否具备 In-context learning 能力)。

架构的选择和具体的微调策略可能比单纯的堆规模更重要。

Tay et al 2023

Some surprising takeaways

The effect of hyperparameters on big LMs can be predicted *before* training!

- Optimizer choice
- Model depth
- Architecture choice

The scaling law based design procedure.

1. Train a few smaller models
2. Establish a scaling law (e.g. ADAM vs SGD scaling law)
3. Select optimal hyperparam based on the scaling law prediction.

One important use of scaling laws

Q: Do we need more data or bigger models?

Clearly, lots of data is wasted on small models

“Lots of data is wasted on small models”。小模型的“脑容量”有限，一旦填满了，再多的数据也学不 进去了。这时候如果不增大模型，纯粹堆数据是浪费算力。

Joint data-model scaling laws describe how the two relate

From Rosenfeld+ 2020,

$$Error = n^{-\alpha} + m^{-\beta} + C$$

From Kaplan+ 2020

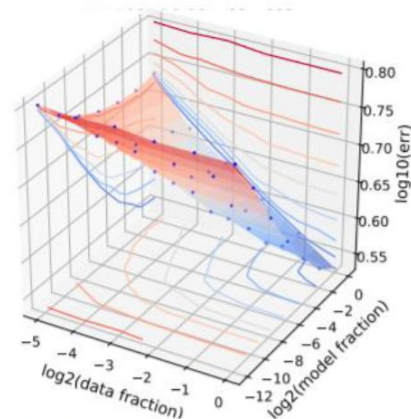
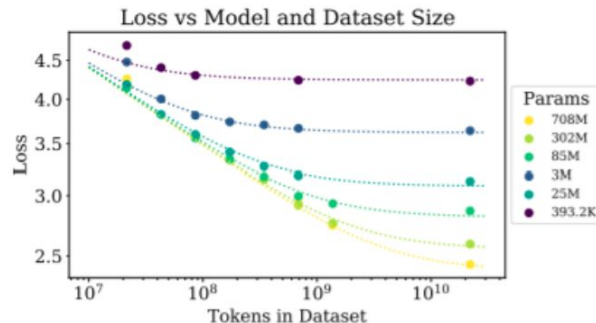
$$Error = [m^{-\alpha} + n^{-1}]^{\beta}$$

Provides surprisingly good fits to model-data joint error.

数据不够带来的误差 ($n^{-\alpha}$)。

模型太笨带来的误差 ($m^{-\beta}$)。

任务本身太难导致的不可避免的误差 (C , Irreducible Error)。



(a) Wiki103 error (cross entropy) landscape.

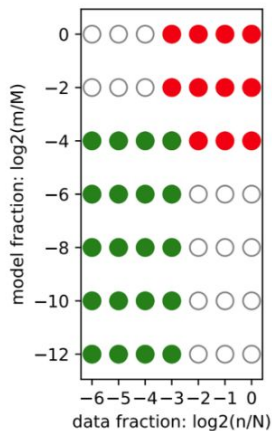
曲面几乎完美地穿过了所有的红点。尽管这些公式是“凭空想出来的(ad hoc)”，但它们对实际误差的拟合令人惊讶地准确

Model-data joint scaling is accurate

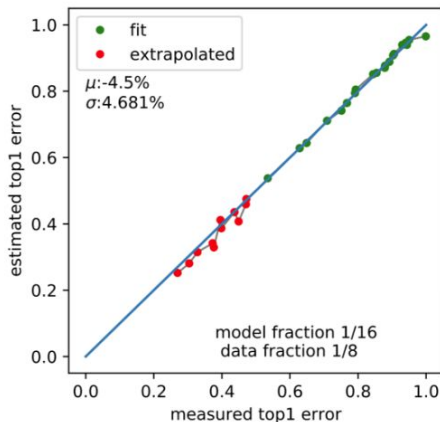
From Rosenfeld – fit scaling exponents on small data, small models. Predict rest.

绿色/白色圆圈 (Fit): 研究人员只训练了这些位于左下角的小模型，使用很少的数据。这就是用来拟合公式中 α , β , C 参数的“训练集”。

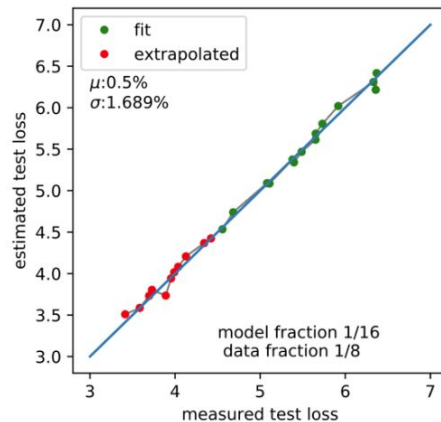
红色圆圈 (Extrapolated): 这些代表更大、更昂贵的模型和数据组合。研究人员**没有**用这些数据来拟合公式，而是用拟合好的公式去**预测**这些红点的性能。



(a) Illustration.



(b) Extrapolation on ImageNet



(c) Extrapolation on WikiText-103.

无论是图像任务 ((b) ImageNet) 还是语言任务 ((c) WikiText-103)。所有的红点(外推预测点)都紧密地贴合在对角线上。这意味着预测误差极小。讲座中提到，这是在仅仅使用了 **1/16 的模型规模** 和 **1/8 的数据量** 的情况下做出的精准预测。

Trading off data size and model size: optimize $n^{-\alpha} + m^{-\beta} + C$ with your costs.

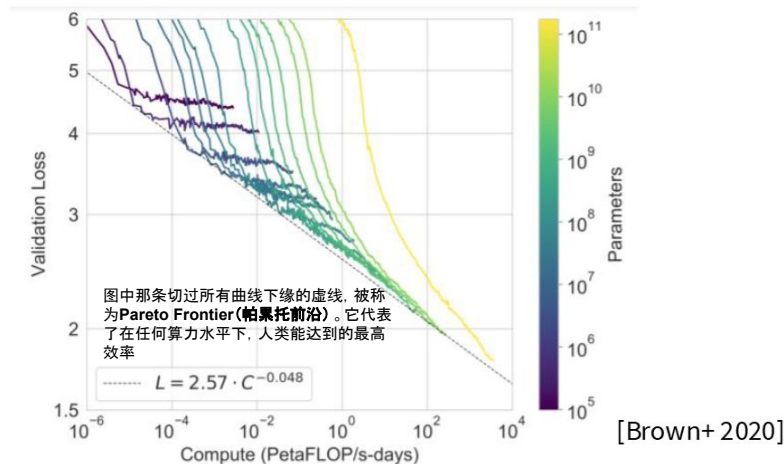
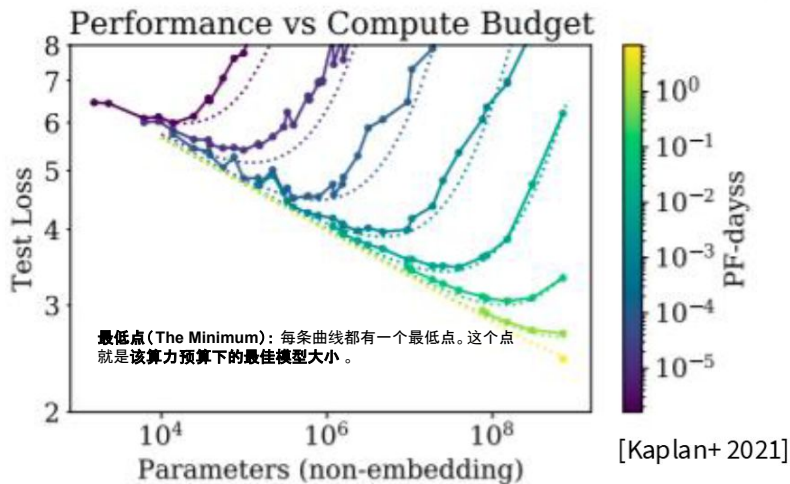
Compute tradeoffs.

Q: what about other resources? Compute vs performance?

算力预算固定 (Fixed Compute Budget)**的情况下, 如何进行最关键的资源权衡: 是应该训练一个很大的模型但只跑很少的步数 (Under-trained), 还是训练一个很小的模型但跑很久 (Well-trained)

For a fixed compute budget...

Big model that's undertrained vs small model that's well trained?



策略 A (左侧): 造一个很小的模型。因为模型小, 计算快, 这笔钱够你把模型训练几十个 Epoch, 直到它滚瓜烂熟。

Scaling laws let us navigate this tradeoff

策略 B (右侧): 造一个巨大的模型。因为模型大, 计算慢, 这笔钱可能只够你让模型看一遍数据 (甚至还没看完), 训练就得停了

不要凭感觉决定模型大小。你应该根据你的总算力预算 (有多少张卡, 跑多久), 在左图的曲线中找到那个最低点, 从而确定出精准的参数数量 (比如 67B 或 175B), 这样才能不浪费任何一点算力。

Caution – ‘Optimal’ scaling laws are hard to get

Rosenfeld, Kaplan both predict relationship of data, model and perf.

这张幻灯片讲述了 AI 扩展定律发展史上的一个著名的“反转”时刻: OpenAI 的 Kaplan 定律 vs. DeepMind 的 Chinchilla 定律。

它揭示了一个残酷的教训: 即使你有了扩展定律的公式, 如果你拟合得不够精准, 你给出的工程建议可能是完全错误的。

来源: Kaplan vs. Chinchilla

比较了两组关于“最佳模型大小”的预测曲线:

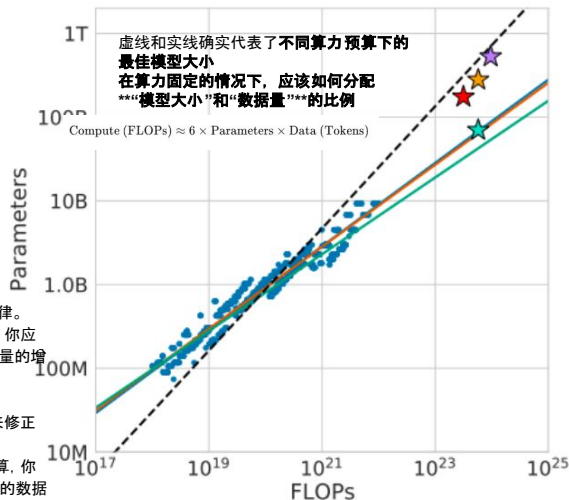
黑色虚线 (Kaplan et al 2020): 这是 OpenAI 最早提出的扩展定律。

- **预测:** 它非常陡峭。这意味着随着算力(横轴)增加, 你应该把绝大部分资源用来把模型做大(纵轴), 而数据量的增加相对较少。
- **潜台词:** “大数据不重要, 模型大才是硬道理。”

彩色实线 (Approach 1, 2, 3 - Chinchilla): 这是 DeepMind 后来修正后的扩展定律。

- **预测:** 它们比较平缓。这意味着对于同样的算力预算, 你应该训练一个相对较小的模型, 但是给它喂多得多的数据。
- **潜台词:** “我们以前搞错了, 数据和模型应该 1:1 同步增长 (即著名的 20 Tokens per Parameter)。”

Chinchilla [Hoffman et al] argue these fits are quite off.



- Approach 1
- Approach 2
- Approach 3
- - - Kaplan et al (2020)

- ★ Chinchilla (70B)
- ★ Gopher (280B)
- ★ GPT-3 (175B)
- ★ Megatron-Turing NLG (530B)

图中的“星星”: 历史的教训

图中的星星代表了当时真实存在的著名大模型, 它们的位置讲述了一个关于“资源浪费”的故事:

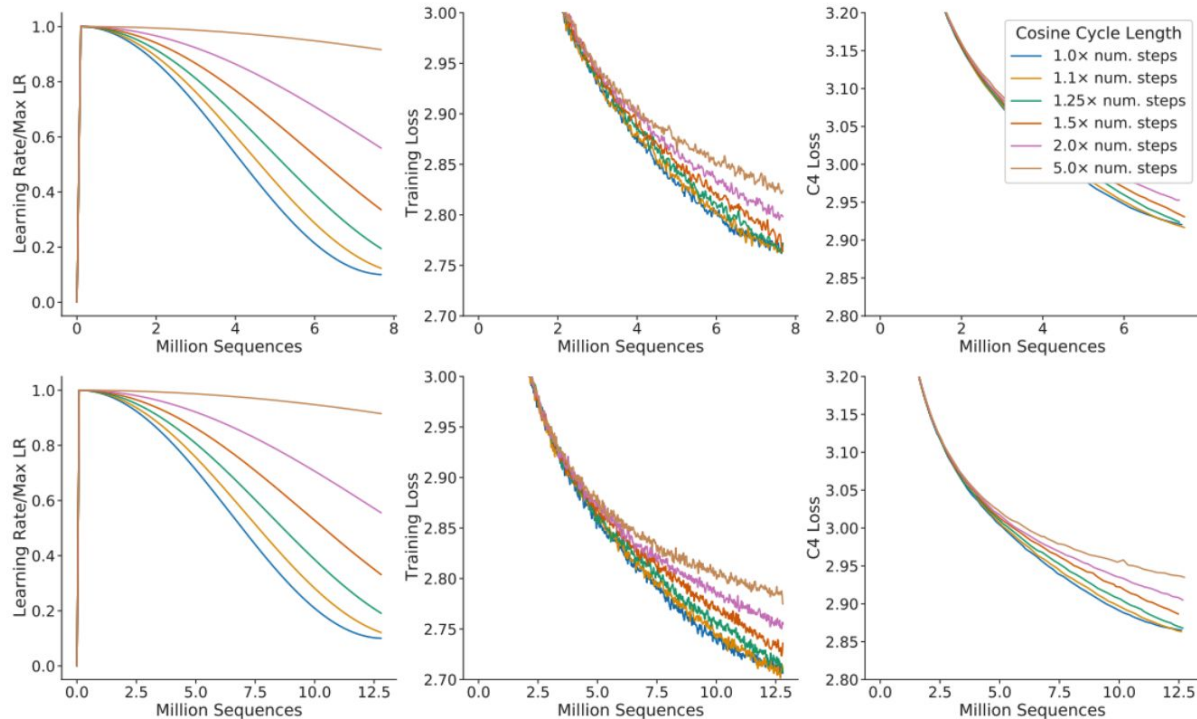
- **紫色星星 (MT-NLG 530B) & 橙色星星 (Gopher 280B):**
 - 这两个模型位于图的上方, 靠近 Kaplan 的黑色虚线。
 - 这说明它们是按照旧的 Kaplan 定律设计的——**过度追求参数量**。结果就是模型太大、太笨重, 训练数据没跟上 (Undertrained), 导致算力利用率不高。
- **青色星星 (Chinchilla 70B):**
 - 这个模型位于彩色实线上。
 - **奇迹:** 尽管它只有 70B 参数 (比 Gopher 小 4 倍), 但因为它用了更多的数据训练 (更符合最优定律), 它的性能打了 280B 的 Gopher。
 - **结论:** 这证明了 Kaplan 的预测是“Quite off” (偏差很大)。

Hoffman+ 2022

Main difference – accounting for LR schedules

讲座中揭示了导致 Kaplan 预测失误的一个关键技术细节——**学习率调度 (Learning Rate Schedule)**。

- **错误做法:** Kaplan 团队在做实验时, 可能通过“截断”并未跑完的 Cosine 学习率模型来估算性能。
- **正确做法:** Chinchilla 团队指出, Cosine 学习率必须完整跑完 (Cool down) 才能得到真实的最佳性能。



“Optimal scaling laws are hard to get” (最优扩展定律很难搞对)。如果你的 Scaling Law 拟合歪了, 你可能会浪费数千万美元去训练一个像 Gopher 那样“虚胖”的模型, 而不是像 Chinchilla 那样“精壮”的模型。这就是为什么现在业界都遵循 Chinchilla Scaling Laws (模型参数与训练 Token 数大约 1:20 的比例) 的原因

Chinchilla in depth – 3 methods

DeepMind 是如何得出 Chinchilla Scaling Law (即模型和数据应 1:1 扩展) 的结论的。
用了三种完全不同的 统计方法 来估算同一个东西。结果发现, 三种方法 给出的答案惊人地一致。

1. 表格核心含义: a 和 b 是什么?

表格中的 a 和 b 是指导算力分配的最关键参数:

- a (Model Coefficient): 当你的总算力 C 增加时, 你的模型参数量 (N_{opt}) 应该增加多少? ($N_{opt} \propto C^a$)
- b (Data Coefficient): 当你的总算力 C 增加时, 你的训练数据量 (D_{opt}) 应该增加多少? ($D_{opt} \propto C^b$)
- 约束条件: 因为 $C \approx 6ND$, 所以在数学上 $a + b$ 必须等于 1。

Approach	Coeff. a where $N_{opt} \propto C^a$	Coeff. b where $D_{opt} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

Kaplan: $a=0.73$, $b=0.27$ 。

- 这意味着算力每增加一点, 73% 的资源应该用来把模型做大, 只有 27% 用来增加数据。

The chinchilla authors suggest 3 ways of fitting scaling laws – we'll go over each.

They mostly (minus method 3) suggest similar constants. More on this later..

结论 $a = b = 0.5$. 模型变大和数据变多同等重要 (各占 50%), 也就是著名的 1:1 扩展比例

Method 1 – minimum over runs.

Similar to the FLOPS figure on Kaplan –
the minimum over the union of all training curves is a power law.

方法 1: 训练曲线包络线

(Minimum over training curves)

- **做法:** 固定模型大小, 看不同数据量 (Tokens) 下的 Loss. (因为 paramters 固定)
- **结果:** $a=0.50$, $b=0.50$.
- **含义:** 模型和数据应该等比例增长. 算力翻 10 倍, 模型扩大 $\sqrt{10}$ 倍, 数据也扩大 $\sqrt{10}$ 倍.

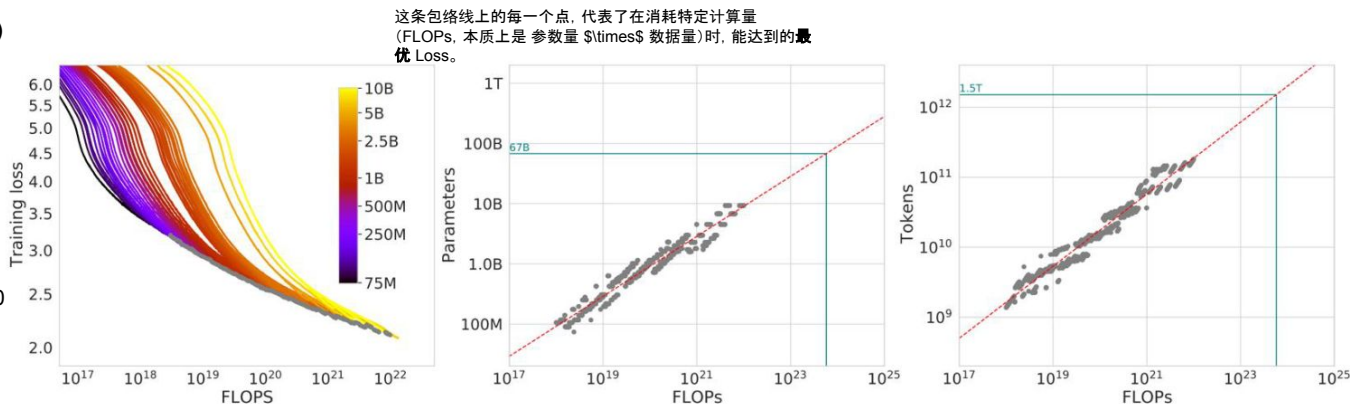


Figure 2 | **Training curve envelope.** On the left we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Method 2 - IsoFLOPS

- 做法: 固定算力预算(比如只给你 100 个 GPU 小时), 在这个预算下尝试各种模型大小, 找到 Loss 最低的那个点。
- 最低点画出parameters 或token vs Flops
- 结果: $a=0.49$, $b=0.51$ 。

Pick a range of FLOP budgets, vary the total parameter count, take the min over these convex shapes. The minima form a power law.

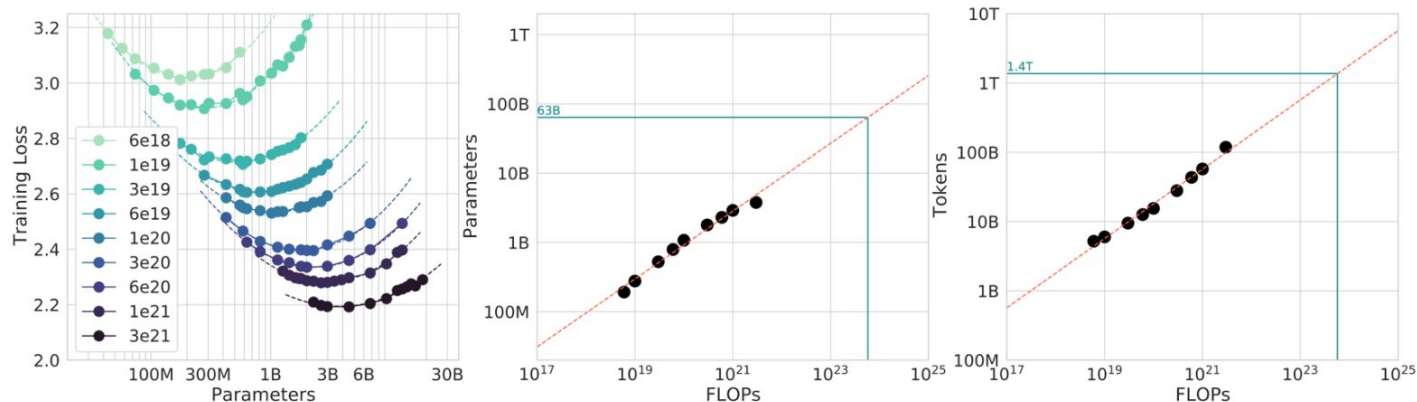


Figure 3 | **IsoFLOP curves.** For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of Gopher.

Method 3 – Joint fits

Run a bunch of models on the size-data grid. Use least squares to fit a joint scaling law

From Rosenfeld+ 2020,

$$\text{Error} = n^{-\alpha} + m^{-\beta} + C$$

方法 3: 参数化损失建模 (Parametric modelling of the loss)

- **做法:** 也就是之前提到的拟合 $\text{Error} = n^{-\alpha} + m^{-\beta} + C$ 这种复杂公式。
- **结果:** $\alpha=0.46, \beta=0.54$ 。
- **讲座趣闻 (Trivia):** 讲座中提到, 这个方法原本的结果 (0.46) 和其他两个稍微有点偏差。后来 Epoch AI 的人去复现时发现, DeepMind 原作者在曲线拟合 (Curve Fitting) 时犯了一个统计学小错误 (curve fitting residual is not 0 mean centered)。修正后, 这个方法的结果也变成了 0.5/0.5, 证明了真理的唯一性

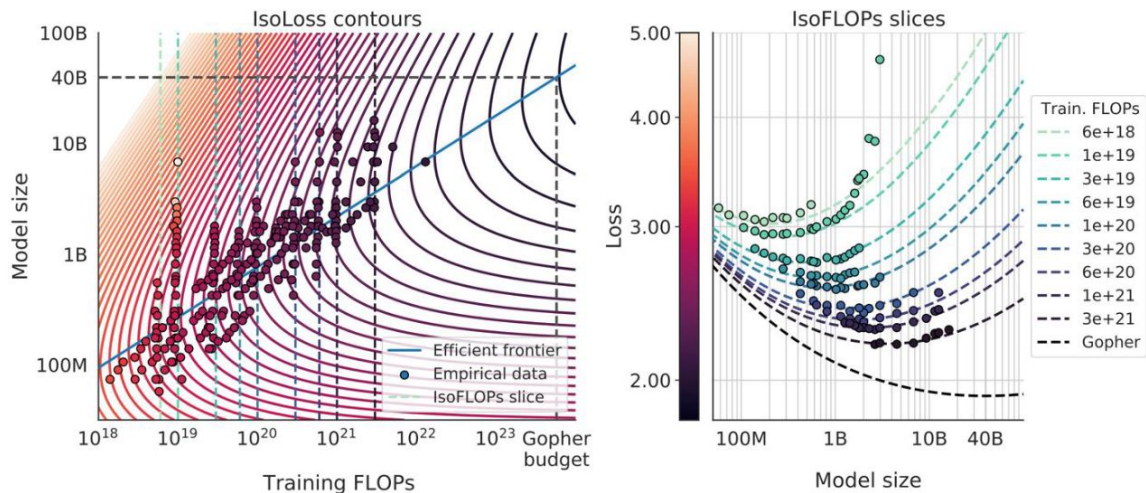


Figure 4 | **Parametric fit.** We fit a parametric modelling of the loss $\hat{L}(N, D)$ and display contour (**left**) and isoFLOP slices (**right**). For each isoFLOP slice, we include a corresponding dashed line in the left plot. In the left plot, we show the efficient frontier in blue, which is a line in log-log space. Specifically, the curve goes through each iso-loss contour at the point with the fewest FLOPs. We project the optimal model size given the *Gopher* FLOP budget to be 40B parameters.

Important note – train-optimal may not be what you want

Chinchilla aims to tell you what gives the best model for fixed training compute..

But most of the compute in a real deployment is inference.. So we should ‘over’ train

Chinchilla Scaling Law (训练最优) 的局限性, 以及为什么现在的模型 (如 Llama 3) 会故意打破这个定律。

- **GPT3** – 2 tokens / param
- **Chinchilla** – 20 tokens / param
- **LLaMA65B** – 22 tokens / param
- **Llama 2 70B** – 29 tokens / param
- **Mistral 7B** – 110 tokens / param
- **Llama 3 70B** – 215 tokens / param

为了节省推理成本 (Inference Cost), 你应该“过度训练 (Over-train)”小模型, 而不是遵循 Chinchilla 的最优比例

Chinchilla Scaling Law 依然是物理学真理, 但在商业决策上, 我们往往选择故意违背它

The more usage we expect, the more it becomes worth it to pay the upfront cost

Recent example for different (diffusion) models

Methods like IsoFLOPS are pretty easy to execute, and our group has replicated these results

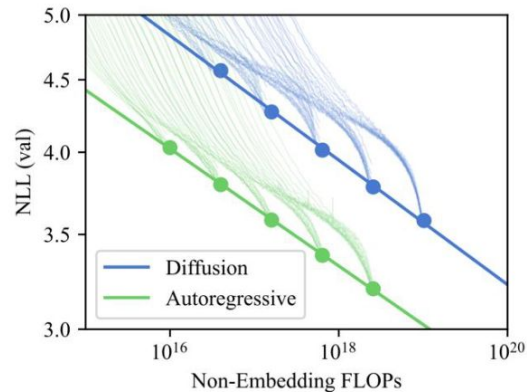
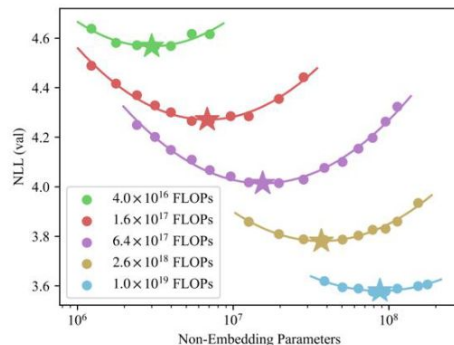
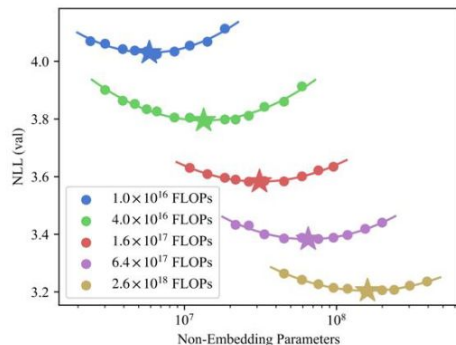


Figure 5: IsoFLOP profiles for autoregressive models (left) and diffusion models (right).

Gulrajani+ 2023.

Apply well on other (new) models and environment as well

Scaling laws for models and compute

Log-linearity extends to model parameters and compute!

Lets us set the following based on small models

- Pick optimizer
- Pick architecture and model sizes

Also lets us make smart resource tradeoffs

- Big models vs more data?

Recap: scaling laws – surprising and useful!

- **Data scaling:** understand how data affects models, clean theory
- **Model scaling:** dramatically reduce costs for training
- **Scaling as prediction:** understand what problems can be ‘brute forced’

Scaling Law 2 - case study and Details

Motivation today

What is the best practices for scaling and hparam tuning LMs?

- Does chinchilla's approach to scaling actually work?
- Can we save compute when training and fitting these things?
- Should we be picking particular architectures / parametrizations to scale nicely?

Log-log plot, isoflop-token trade-off; optimal learning rate

Scaling in practice

The newest model we talked about with scaling details - 2022



Training Compute-Optimal Large Language Models

Jordan Hoffmann*, Sebastian Borgeaud*, Arthur Mensch*, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals and Laurent Sifre*

*Equal contributions

What about more recently?



Cerebras-GPT: Open Compute-Optimal Language Models Trained on the Cerebras Wafer-Scale Cluster

Nolan Dey, Gurpreet Gosal, Zhiming (Charles) Chen, Hemant Khachane, William Marshall, Ribhu Pathria, Marvin Tom, Joel Hestness
Cerebras Systems



DeepSeek LLM Scaling Open-Source Language Models with Longtermism

2023

2024



The Llama 3 Herd of Models

Scaling becomes
secretes, much
sparser paper.
Deepseek and
miniCPM becomes
gold standard for
modern scaling law

MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies

Shengding Hu¹, Yuge Tu², Xu Han¹, Chaoqun He¹, Ganqu Cui¹, Xiang Long², Zhi Zheng², Yewei Fang², Yuxiang Huang¹, Weilin Zhao¹, Xinrong Zhang¹, Zheng Leng Thai¹, Kaihuo Zhang², Chongyi Wang², Yuan Yao¹, Chongyang Zhao¹, Jie Zhou², Jie Cai², Zhongwu Zhai², Ning Ding¹, Chao Jia², Guoyang Zeng², Dahai Li², Zhiyuan Liu^{1*}, Maosong Sun¹

¹Department of Computer Science and Technology, Tsinghua University.

²Modelbest Inc.

2024



MiniMax-01: Scaling Foundation Models with Lightning Attention

Maximum update parametrization – in depth

Optimal LR
stays stable

Recall – the maximum update parametrization makes appealing claims

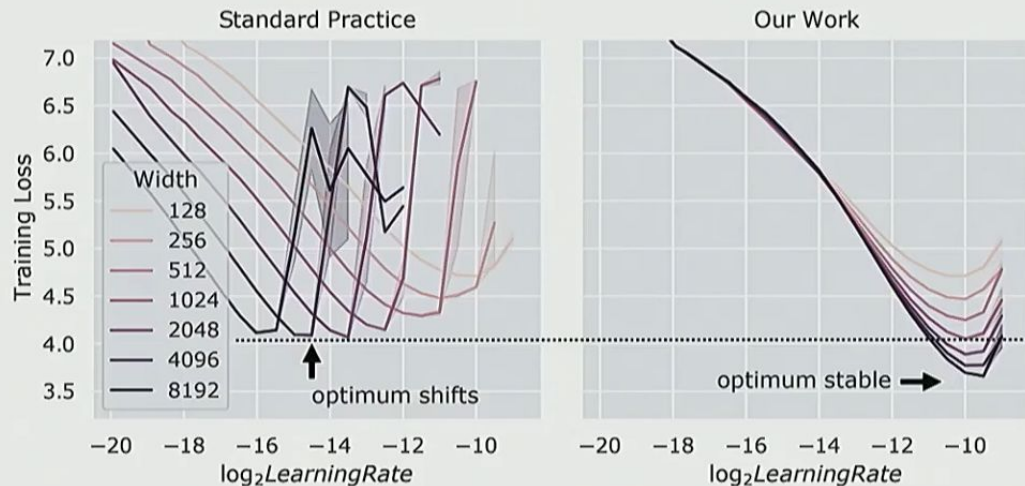


Table 2. μP function for a model M' that is r times the widths of M . If a parameter tensor has 2 dimensions that goes infinite when the model width goes infinite, it is “matrix-like” (e.g., a fully-connected hidden layer); if the number is 1 or 0, it belongs to the “others” class. Note that embedding layers are “others”. “Output” means the layer that maps an infinite dimension to a finite dimension, which is the word decoding layer (*lm_head*) in Transformers. A multiplier is a constant multiplied by a parameter tensor, which has a similar function to softmax temperature.

Hyperparameter (weight)	M	$M' \sim r$
AdamW learning rate (matrix-like)	l	l/r
AdamW learning rate (others)	l	l
Initialization variance (matrix-like)	σ	σ/r
Initialization variance (others)	σ	σ
Multiplier (output)	τ	τ/r
Multiplier (others)	τ	τ

以下几个ppt会讨论MuP在几个现实中的大模型中的应用

Scale-invariant hyperparameter tuning would be very nice

How does it work, and does it work in practice?

Stanford

CerebrasGPT

Case study on recent models:

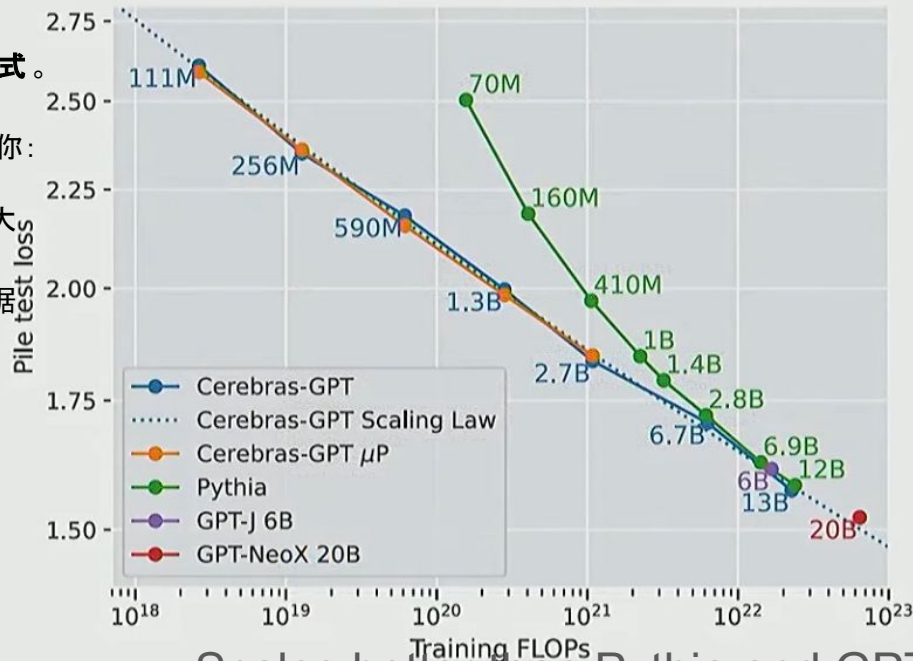
CerebrasGPT, miniCPM, Deepseek

CerebrasGPT – 0.1 to 13B models trained with the Chinchilla recipe.

Chinchilla Recipe 最优解公式。

如果你拥有一定的算力
(FLOPs), 这个“配方”能告诉你:

- 你的模型应该做成多大
(多少亿参数)?
- 你需要喂给它多少数据
(多少万亿 token)?



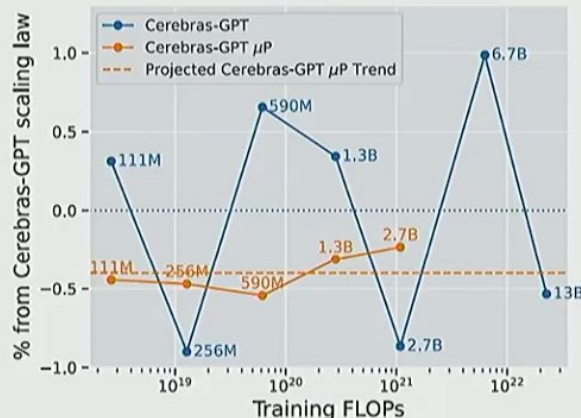
Scales better than Pythia and GPT-J

Core finding – using μ P parametrization makes scaling more stable

Hyperparam scaling strategy

Cerebras GPT authors find more predictable scaling from μ P parametrization

Dash line 是预测的学习率, 橙色的更加 stable, 更加接近预测学习率



Without μ P, big oscillations

Figure 5: Percentage loss increase relative to Cerebras-GPT scaling law plotted against training FLOPs.

We train a set of Cerebras-GPT models using μ P. We follow the μ Transfer approach by first tuning hyperparameters for a small, 40M parameter μ P model. Then, we transfer the hyperparameters along our μ P scaling law up to a 2.7B parameter model. μ P requires small changes to our baseline Cerebras-GPT models, including adding element-wise activation tensor scaling, adjusting initializers for affected layers, and adding layer-wise learning rates scaling to certain layers. We discuss the benefits we see with μ P in Section 3.3. Refer to Appendix G for our tips to implement μ P and our hyperparameter tuning notes.

muP parametrization

Key change: initialization + per layer LR

Appendix contains a very clear set of differences in parametrizing the model for scaling

Mathematics

MuP的各种参数不一样

Variables	Standard Parameterization (SP)	Maximal Update (μP)
W	A multiplicative or fully-connected weights tensor	
b	A bias weights tensor	
X, Y	Activation tensors: layer input, output, respectively	
$d_{\text{model,base}}$	Proxy (base) model's layer width	
d_{model}	Width of each layer	
d_{head}	Size of each attention head	
embed	Combined token + position embedding function	
η_{base}	The base learning rate (LR): Maximum in training schedule	
σ_{base}	The base initialization standard deviation	
m_{width}	—	Layer width multiplier: $d_{\text{model}}/d_{\text{model,base}}$
m_{emb}	—	Embedding output multiplier

Standard Parameterization (SP)

Maximal Update (μP)

Empirically Tuned Values

$d_{\text{model,base}}$	—	256
η_{base}	Must tune for each model size	6e-3
σ_{base}	0.02	0.08
m_{emb}	—	10.0

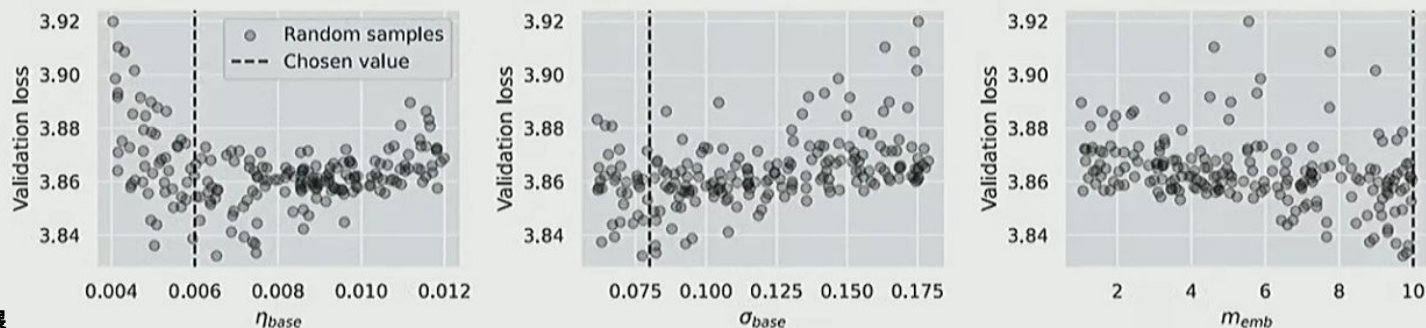
Formulas

Embedding initializer	$W_{\text{emb}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2)$	
Embedding LR	$\eta_{\text{emb}} = \eta_{\text{base}}$	
Embedding output	$Y_{\text{emb}} = \text{embed}(X)$	
LN initializer	$W_{\gamma} \sim 1, b_{\beta} \sim 0$	
LN LR	$\eta_{\text{LN}} = \eta_{\text{base}}$	
Bias initializer	$b \sim 0$	
Bias LR	$\eta_b = \eta_{\text{base}}$	
MHA equation	$\text{softmax}\left(\frac{Q^T K}{\sqrt{d_{\text{head}}}}\right) V$	
QKV weights initializer	$W_{\text{qkv}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2)$	$W_{\text{qkv}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2/m_{\text{width}})$
QKV weights LR	$\eta_{\text{qkv}} = \eta_{\text{base}}$	$\eta_{\text{qkv}} = \eta_{\text{base}}/m_{\text{width}}$
O weights initializer	$W_o \sim N_{\text{trunc}}(0, \frac{\sigma_{\text{base}}^2}{2 \cdot n_{\text{layers}}})$	$W_o \sim N_{\text{trunc}}(0, \frac{\sigma_{\text{base}}^2}{2 m_{\text{width}} \cdot n_{\text{layers}}})$
O weights LR	$\eta_o = \eta_{\text{base}}$	$\eta_o = \eta_{\text{base}}/m_{\text{width}}$
ffn1 weights initializer	$W_{\text{ffn1}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2)$	$W_{\text{ffn1}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2/m_{\text{width}})$
ffn1 weights LR	$\eta_{\text{ffn1}} = \eta_{\text{base}}$	$\eta_{\text{ffn1}} = \eta_{\text{base}}/m_{\text{width}}$
ffn2 weights initializer	$W_{\text{ffn2}} \sim N_{\text{trunc}}(0, \frac{\sigma_{\text{base}}^2}{2 \cdot n_{\text{layers}}})$	$W_{\text{ffn2}} \sim N_{\text{trunc}}(0, \frac{\sigma_{\text{base}}^2}{2 m_{\text{width}} \cdot n_{\text{layers}}})$
ffn2 weights LR	$\eta_{\text{ffn2}} = \eta_{\text{base}}$	$\eta_{\text{ffn2}} = \eta_{\text{base}}/m_{\text{width}}$
Output logits multiplier	$Y_{\text{logits}} = W_{\text{unemb}} X$	$Y_{\text{logits}} = W_{\text{unemb}} X / m_{\text{width}}$

Setting the empirical values

muP is combined with aggressive scaling for hyperparameter optimization.

We tune μP hyperparameters on a 40M parameter proxy model, and $\mu Transfer$ those hyperparameters to our models with 111M–2.7B parameters. Figure 13 shows the results of a 200 sample random hyperparameter search on a 40M parameter proxy model ($d_{\text{model}} = d_{\text{model,base}} = 256$, $n_{\text{layers}} = 32$, $d_{\text{head}} = 128$) trained on 600M tokens with a batch size of 131k tokens. From this sweep we obtained the following tuned hyperparameters for our μP models: $\eta_{\text{base}} = 6e-3$, $\sigma_{\text{base}} = 0.08$, $m_{\text{emb}} = 10$. These hyperparameters also closely match those used by Yang et al. (2021).



用小的 40M 的模型做超参数 search，然后扩展到大模型上面使得超参数 stable (取使得 loss 最小的超参数)

Figure 13: Results from random hyperparameter search on 40M parameter μP proxy model.

Generally stable hyperparameters.

Stanford

MiniCPM

MiniCPM (2024) – new small, high-perf LM from Tsinghua group.

First extensive
scaling paper

MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies

Shengding Hu¹, Yuge Tu², Xu Han^{1*}, Chaoqun He¹, Ganqu Cui¹, Xiang Long²,
Zhi Zheng², Yewei Fang², Yuxiang Huang¹, Weilin Zhao¹, Xinrong Zhang¹,
Zheng Leng Thai¹, Kaihuo Zhang², Chongyi Wang², Yuan Yao¹,
Chenyang Zhao¹, Jie Zhou², Jie Cai², Zhongwu Zhai², Ning Ding¹,
Chao Jia², Guoyang Zeng², Dahai Li², Zhiyuan Liu^{1*}, Maosong Sun¹

¹Department of Computer Science and Technology, Tsinghua University.

²Modelbest Inc.

Goal: train relatively small model, train lots of good small models

Careful, extensive scaling computations + muP to stabilize and simplify scaling

Stanford

MiniCPM

High performance 1-2.5 B parameter models. These models beat most out 2Bs and match many modern 7B models.

首先的到一个好的 model

Model	C-Eval	CMMLU	MMLU	HumanEval	MBPP	GSM8K	MATH
Llama2-7B	32.42	31.11	44.32	12.20	27.17	13.57	1.80
Qwen-7B	58.96	60.35	57.65	17.07	42.15	41.24	5.34
Deepseek-7B	42.82	44.45	47.82	20.12	41.45	15.85	1.53
Mistral-7B	46.12	42.96	62.69	27.44	45.20	33.13	5.00
Gemma-7B	42.57	44.20	60.83	38.41	50.12	47.31	6.18
Llama2-13B	37.32	37.06	54.71	17.07	32.55	21.15	2.25
MPT-30B	29.34	32.09	46.56	21.95	35.36	10.31	1.56
Falcon-40B	40.29	41.57	53.53	24.39	36.53	22.44	1.92
TinyLlama-1.1B	25.02	24.03	24.3	6.71	19.91	2.27	0.74
Qwen-1.8B	49.81	45.32	43.37	7.93	17.8	19.26	2.42
Qwen1.5-1.8B	55.00	50.85	43.81	5.49	24.82	26.16	3.25
Gemini Nano-3B	-	-	-	-	27.20	22.80	-
StableLM-Zephyr-3B	30.34	30.89	45.90	35.37	31.85	52.54	12.12
Phi-2(2B)	23.37	24.18	52.66	47.56	55.04	57.16	3.50
Gemma-2B	29.26	28.56	38.49	24.39	29.74	16.83	3.34
MiniCPM-1.2B	49.14	46.81	49.63	44.51	32.75	31.77	10.60
MiniCPM-2.4B	51.13	51.07	53.46	50.00	47.31	53.83	10.24

Stanford

Technique 1: muP to stabilize scaling

在scale model 前, 先保证超参数
能很好的 scale

Scale_emb = 12, scale_depth = 1.4, init_std = 0.1, lr = 0.01

Name	Specific Operation
Embedding Output Scaling	Multiply the output of the embedding by <i>scale_emb</i>
Residual Connection Scaling	Scale the increment at each residual connection in each layer by $scale_depth / \sqrt{num_layers}$
Initialization of Tensors	Set the initialization standard deviation of each two-dimensional tensor parameter to $init_std / \sqrt{d_m / d_{base}}$, and set other parameters' initialization to 0.1
Learning Rate Scaling of Tensors	Adjust the learning rate of each two-dimensional tensor parameter to $1 / (d_m / d_{base})$ times the learning rate of other parts (or the overall learning rate)
LM Head Scaling	Adjust the output logits to $1 / (d_m / d_{base})$ times the original value

Table 7: List of operations used when applying tensor program techniques.

c.f. CerebrasGPT – Scale_emb = 10, lr=6e-3, init_base = 0.08

Stanford

Scaling recipe / strategy

Use muP for initialization, fix the aspect ratio, scale up the overall model size.

第二步, 就是选个学习率尽量不变, 然后 scale up model 大小

Name	N (B)	d_m	d_{ff}	d_h	n_h	L
9M	0.009	320	800	64	5	8
30M	0.036	512	1280	64	8	12
70M	0.066	640	1600	64	10	14
0.1B	0.109	768	1920	64	12	16
0.17B	0.166	896	2240	64	14	18
0.2B	0.241	1024	2560	64	16	20
0.5B	0.499	1344	3360	64	21	24

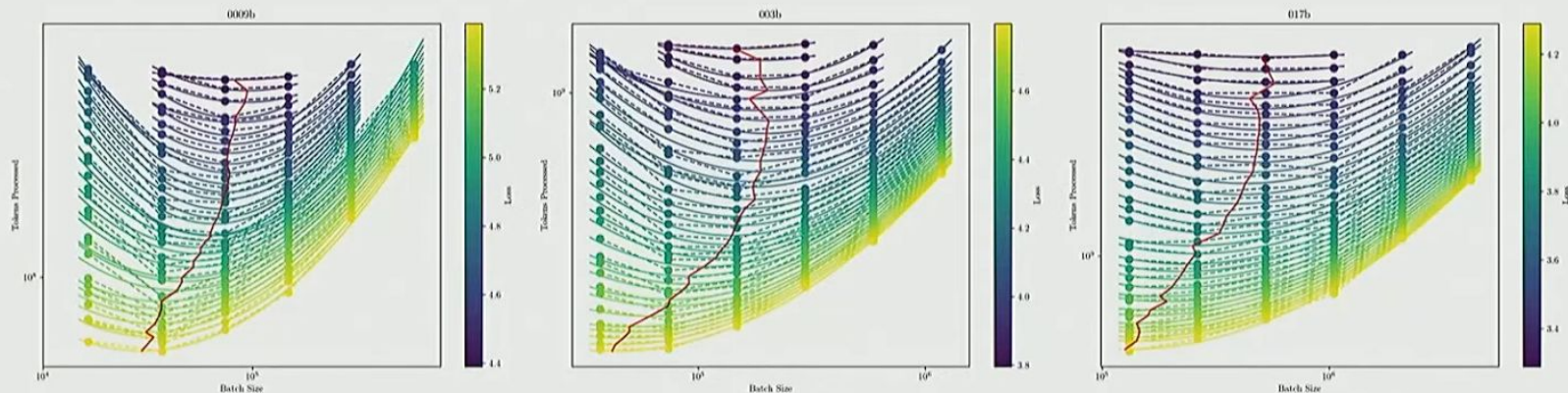
Note that the gap between the largest model here and the actual model they train is ~5x

Optimal batch, LR, token-to-size ratios are directly fitted via scaling analysis

Optimal batch

然后就能找到 optimal batch size (critical batch size - diminishing return point) 和scale的关系, 红线是最小值

Three model sizes (9m, 30m, 170m) as a function of data size (y), batch (x) and loss (col)



Vertical columns of points represent a single training curve (fixed batch, more points).

Red line attempts to identify minimum loss points for each y-value

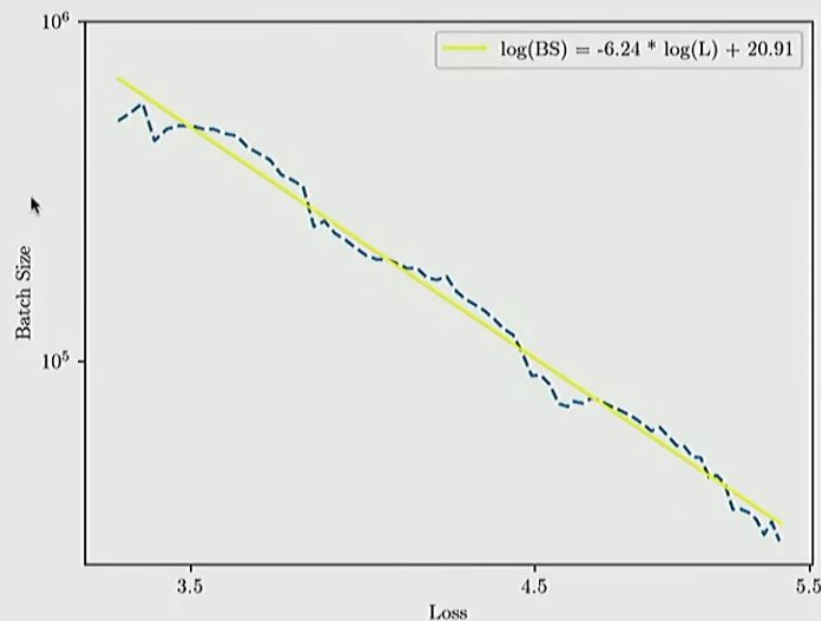
– this is the ‘optimal batch size’ for a model size / dataset size combination.

Stanford

Optimal batch size

We can then follow the Kaplan 2020 analysis and plot optimal batch size vs final loss.

先用target scale 定下
loss (predictable), 然后找
对应的batch size



Fairly clean trend – polynomially increase the batch size as loss decreases.

Optimal LR

According to muP – optimal learning rate should be (roughly) stable. Is it?

尽管loss 超过一定数值, 模型变得不稳定, 但loss 最小值对应的learning rate (optimal learning rate)是一致的

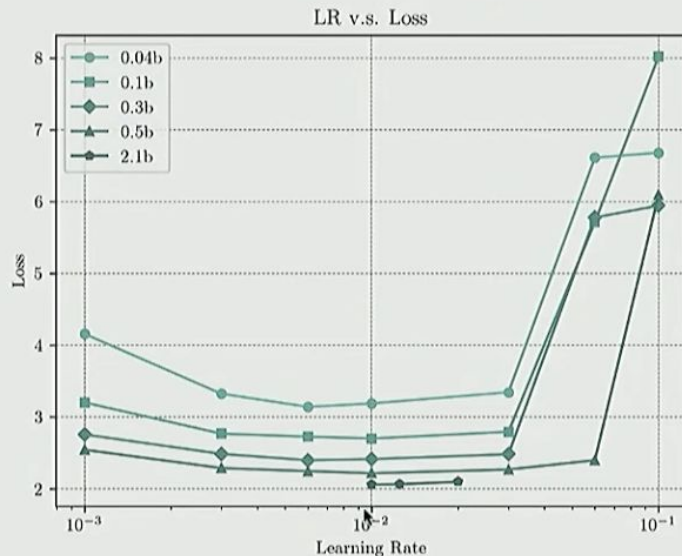
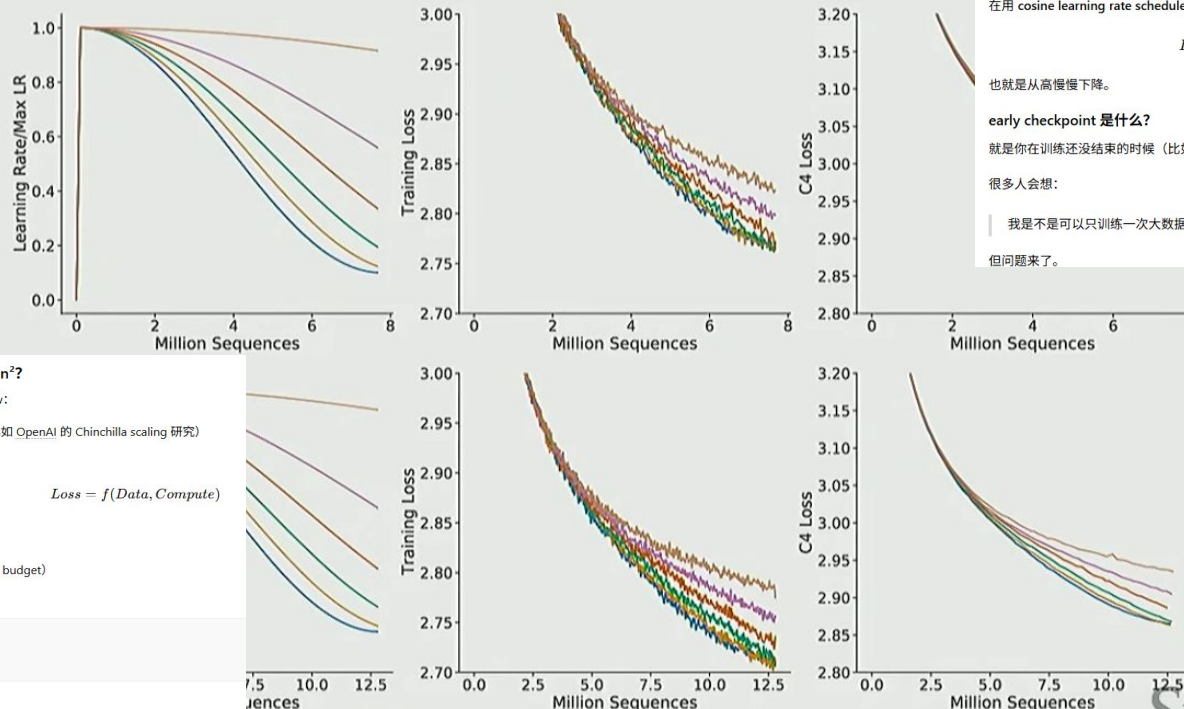


Figure 3: Loss vs Learning Rate. After applying for the Tensor Program, the learning rate shift becomes minimal.

What remains – model size vs data trad

你不能只用一次 cosine learning rate 的训练曲线，去“推断”不同数据规模下的 early checkpoint 行为；
因为不同数据量下，early stop 之后的“冷却时间（cool down）”是不一样的；
所以每个数据规模都必须重新完整训练一次；
因此如果你要同时对“数据规模 × 训练步数”做拟合，实验复杂度会变成 n^2 。

From chinchilla – to fit a scaling law, we need to train from scratch, not just early stop



第四部分：为什么变成 n^2 ？

如果你想拟合一个 scaling law：

例如类似 OpenAI 的论文（比如 OpenAI 的 Chinchilla scaling 研究）

你要研究：

$$Loss = f(Data, Compute)$$

假设你有：

- n 种数据规模
- n 种训练步数（compute budget）

那你理论上要跑：

Code

$n \times n = n^2$ 个实验

因为：

- 每个数据规模
- 都要在多个 compute 下训练

第一部分：early stop + cool down 是什么？

在用 cosine learning rate schedule 训练大模型时，学习率是：

$$LR(t) = LR_{max} \cdot \frac{1}{2} (1 + \cos(\pi t/T))$$

也就是从高慢慢下降。

early checkpoint 是什么？

就是你在训练还没结束的时候（比如 20%、40%、60%）保存一个模型。

很多人会想：

我是不是可以只训练一次大数据模型，然后用它的 early checkpoints 来模拟小数据训练的效果？

但问题来了。

第二部分：数据量变大 → cool down 时间变长

所谓 cool down 时间 是指：

当学习率已经很低的时候，模型还在做微小调整的阶段。

在大数据下：

- 模型能持续看到新的样本
- 梯度噪声更稳定
- 训练曲线更平滑
- 收敛阶段更长

所以：

数据越大，训练后期的“缓慢精细调整阶段”越长

这意味着：

- 小数据模型在 step 100k 收敛
- 大数据模型可能在 100k 还没真正进入最终收敛阶段

所以同一个 cosine schedule 的 early checkpoint：

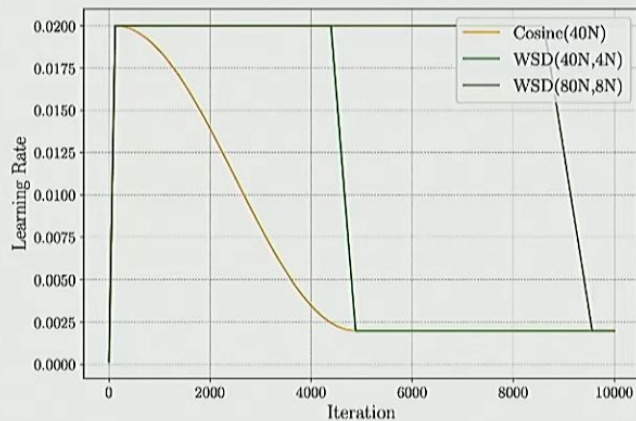
- 对小数据来说是“已收敛模型”
- 对大数据来说只是“中期模型”

因此不能直接对比。

This turns the cost of fitting a scaling law from n to n^2 .. Can we avoid

(partial) solution in miniCPM – WSD learning rate

Instead of cosine, split learning rate into warmup, stable, and decay phases.



所以用另外一种 learning rate scheduling 的方式 (WSD LR) 就是warmup后有一段是 stable 的(flat), 在stable 阶段, 不用考虑数据对learning rate 的影响, 可以做scaling law

In this section, we introduce the utilization of the WSD scheduler as an effective approach to explore the scaling law with linear cost ($O(mC)$). Since WSD scheduler has the advantage of arriving at the optimal loss of Cosine LRS after decaying from stable stage's checkpoints of any step, we are now able to precisely measure the optimal scaling properties without re-training the models from scratch to different amount of tokens, thus making the scaling law measurement much more efficient along the data axis.

We measure the scaling law along the data and model axes by training SLMs of 6 sizes ranging from 0.04B to 2B, each with 6 decayed model starting from checkpoint of 10N to 60N data during the stable training stage. The final loss is evaluated on five held-out evaluation dataset. To potentially compare the loss when the model uses different tokenizer, we take the average of loss by number of bytes instead of number of tokens, following Achiam et al. (2023). The final loss of each pair of data size and model size is shown in the blue lines in Figure 17.

Figure 15: Illustrative comparison between Cosine LRS and WSD LRS. The WSD LRS with different end steps share the same stable training stage.

Stanford

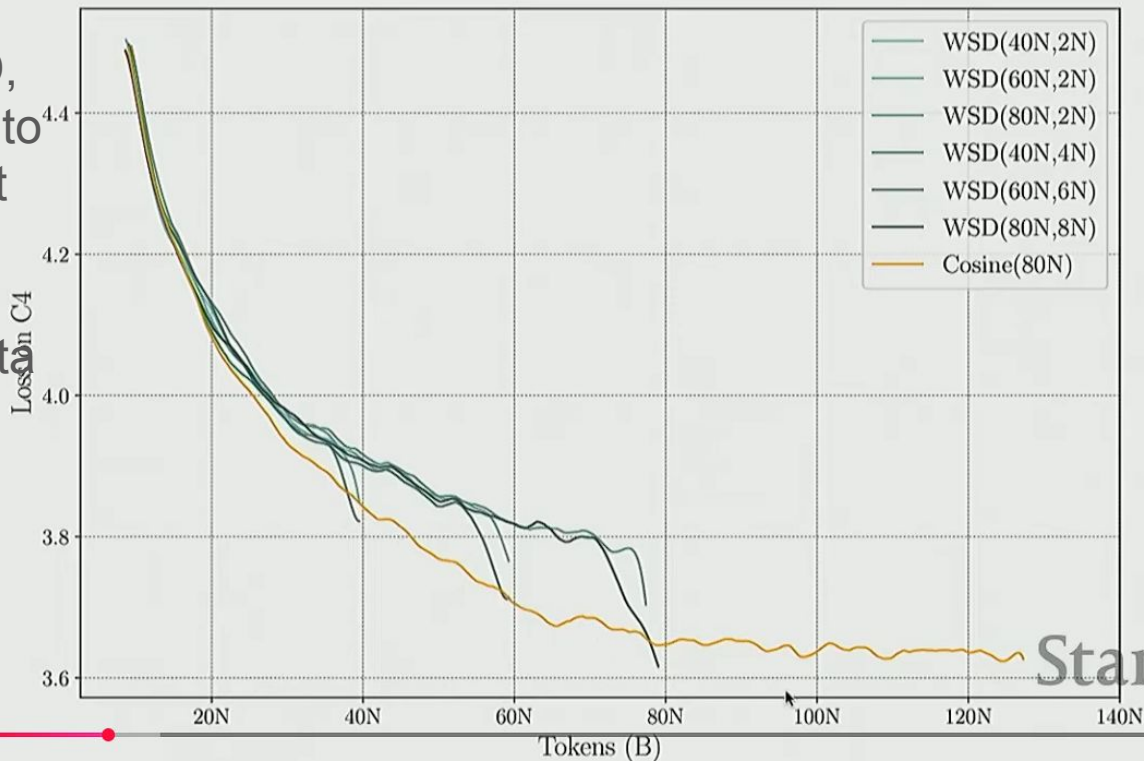
For chinchilla-style analysis, can restart the run at the end of the stable phase.

Subtitles/closed captions [C]

WSD learning rates work well in miniCPM

Slower during the stable phase, rapid loss decay during decay phase. Decay ~ 1

Using WSD,
Don't need to
worry about
termination
point for
different data
size



Stanford

Chinchilla-type analysis

Equipped with the WSD learning rate,
we can now try to find the optimal data-to-model size ratio

Then we fit the losses with model size N and data size D following [Hoffmann et al. \(2022\)](#) using `scipy curvefit` function:

$$L(N, D) = C_N N^{-\alpha} + C_D D^{-\beta} + L_0 \quad (2)$$

The fitted curve along the data axis for each dataset and each checkpoints are shown in orange lines in Figure 17. Then we have the optimal model size N_{opt} , dataset size D_{opt} , given a fixed amount of compute $C = 6ND$ ([Rae et al., 2021](#)) as:

Solve optimal token to data ratio through the

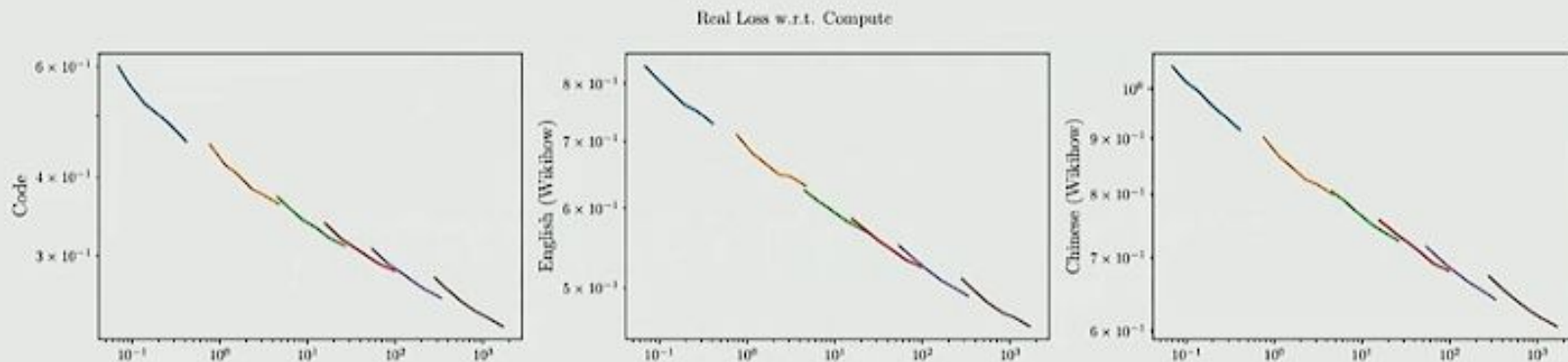
$$\frac{N_{opt}}{D_{opt}} = K^2 \left(\frac{C}{6} \right)^\eta, \quad (3)$$

MiniCPM authors choose method 1 (lower envelope) and method 3 (joint fit)

Chinchlla method 1

Fairly clear (though maybe not linear?) trends

Compute to token ratio near linear



Different colors indicate different models. Their runs suggest relatively low diminishing returns due to data.

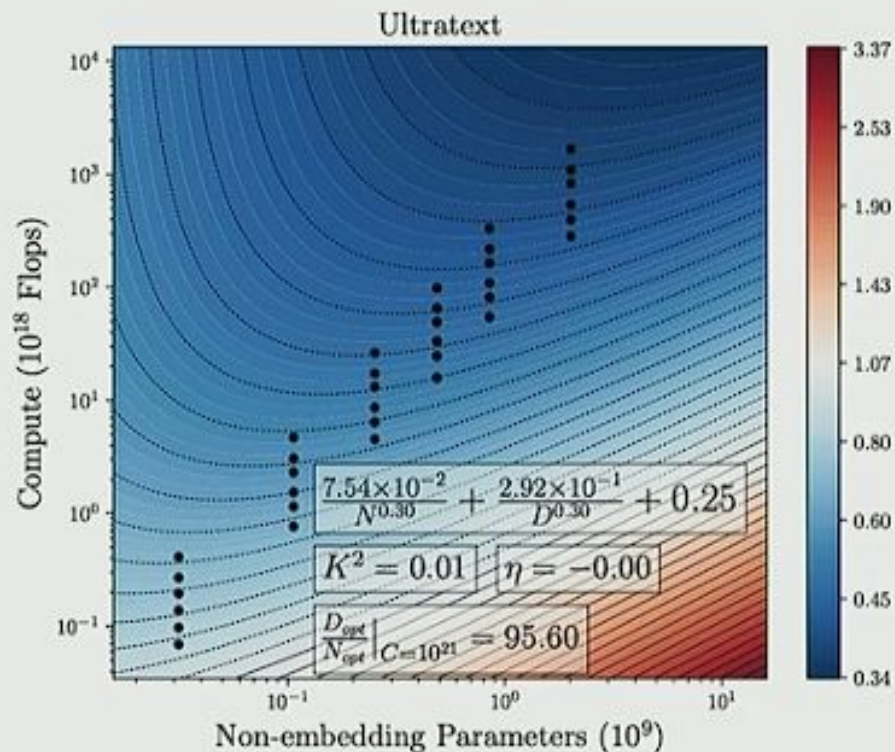
Chinchilla method 3

Their primary scaling approach is the joint fit – they find very high data-model ratios.

Primarily use method 3 to justify

Dots: small scale

Curve: fittings



Stanford

Too high data to model
ratio

用**远超**理论最小需求的海量数据, 去 训练一个
相对较小的模型

Don't see Diminishing Returns (收益递减
效应)

Tiny models with lots of data

The overall data-to-model ratio is very high (192), though they argue LLaMA architectures should have a higher ratio.

As for the concrete data-to-model ratio $\frac{D_{opt}}{N_{opt}}$, we notice that there is a huge gap in compute optimal regime of between ours and Hoffmann et al. (2022) despite that the trend of $\frac{D_{opt}}{N_{opt}}$ with compute C is aligned between ours and theirs. Specifically, the data size should be 192 times larger than the model size on average, as opposed to 20 times in Hoffmann et al. (2022). We note that this aligns with the observation in Section 4.3 and Figure 6.

Note that recent models like LLaMA 3 has *significantly* higher data-to-model ratios, suggesting that with more careful optimization, we might be able to go far beyond the 20*model_size rule of thumb.

为什么 这是好事? (战略意义)

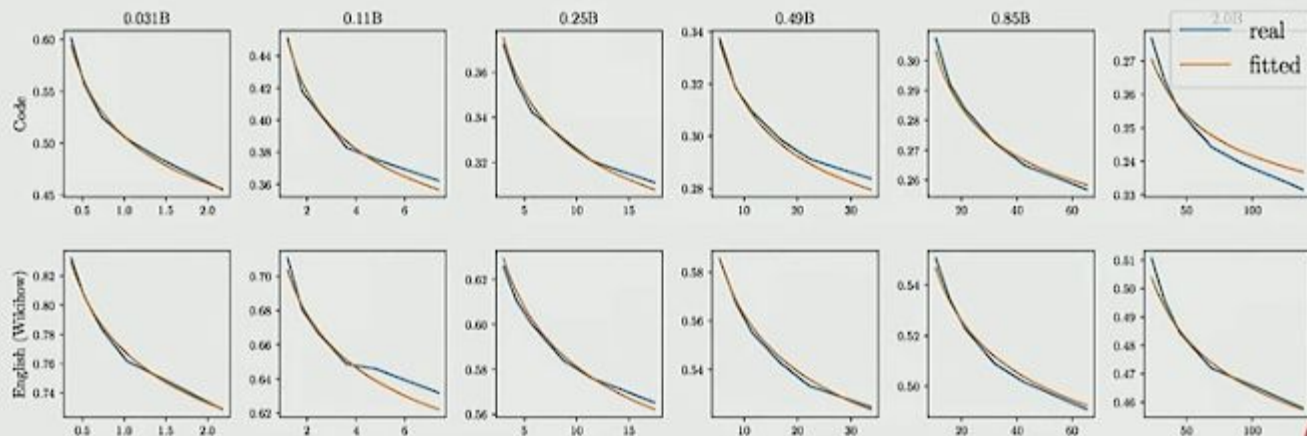
A. 极度降低推理成本 (Inference Efficiency)

- 如果你能把 70B 的智力压缩进一个 7B 的模型. **推理成本直接下降了 10 倍。**

B. 突破“内存墙” (Breaking the Memory Wall)

Scaling curve fits are (generally) good

Overall fits and predictions of models across a large range of sizes is fairly good.



X-axis, number of tokens in billions.

DeepSeek

DeepSeek (2024) – another LM with careful scaling analysis



DeepSeek LLM
Scaling Open-Source Language Models with Longtermism

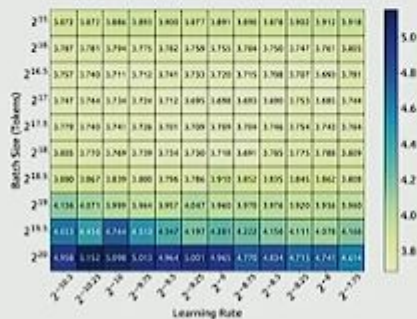
7 and 67B param models – generally high performance compared to other open LM

Run grid with batch size and LR, do the same for small/large model

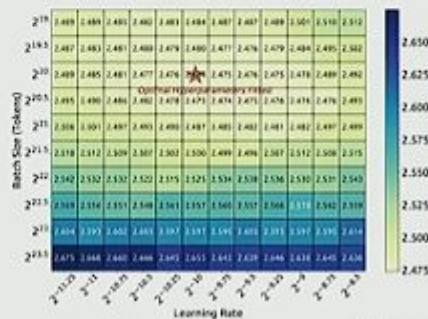
Scaling strategy – batch + LR

Scaling strategy: don't use any muP, directly estimate optimal batch / LR

We initially conducted a grid search for batch size and learning rate on small-scale experiments with a compute budget of $1e17$, and the results of a specific model size (177M FLOPs/token) are illustrated in Figure 2(a). The results demonstrate that the generalization error remains stable across a wide range of choices of batch sizes and learning rates. This indicates that near-optimal performance can be achieved within a relatively wide parameter space.



(a) $1e17$ FLOPs (177M FLOPs/token)



(b) $1e20$ FLOPs (2.94B FLOPs/token)

Stanford

Making the grid for different FLOPs, marking down the star (optimal batch size and LR) - follow scaling behavior

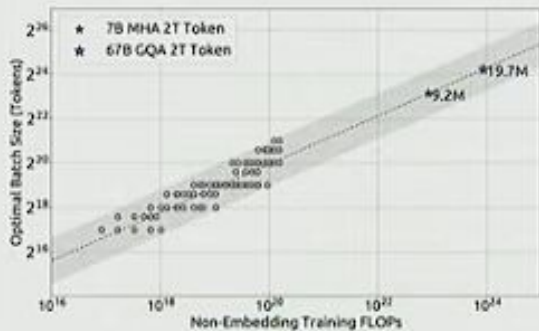
Scaling analysis of learning rates

Small scale runs + collect 'near optimal' (within 0.25% of min) models.

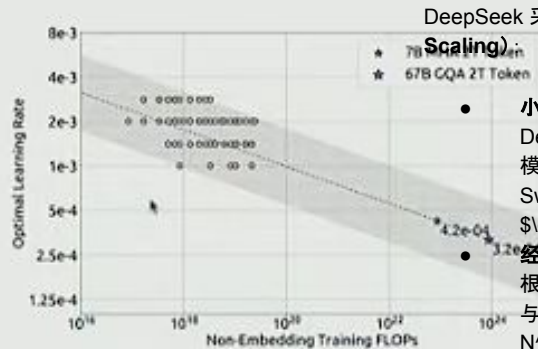
$$\eta_{\text{opt}} = 0.3118 \cdot C^{-0.1250}$$

$$B_{\text{opt}} = 0.2920 \cdot C^{0.3271}$$

(1)



(a) Batch size scaling curve



(b) Learning rate scaling curve

Learning rate fit looks a bit questionable..

DeepSeek 采用的是 **小规模实验 + 启发式缩放 (Heuristic Scaling)**:

- **小模型先行 (Small-scale Proxy Tasks):**

DeepSeek 团队会先在 100M、1B、7B 等不同量级模型上进行充分的超参数 扫频 (Hyperparameter Sweep)。他们记录下每个规模下的最优学习率 η_{opt} 。

- **经验公式外推:**

根据 OpenAI 和 DeepMind 的研究, 最优学习率通常与模型规模 N 呈幂律关系: $\eta_{\text{opt}} \approx a \cdot N^{-b}$ 。DeepSeek 通过拟合这些小实验的数据点来预测 671B (MoE) 或更大模型所需的最优 LR。

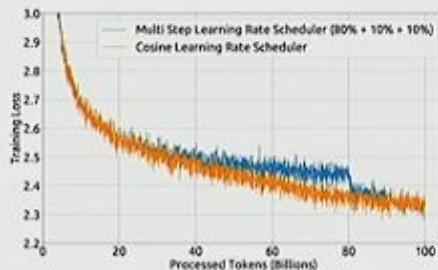
- **余弦退火与 预热 (LR Scheduler):**

他们通常使用带有较长 Warm-up 阶段的 **Cosine Annealing**。在 2026 年的实践中, DeepSeek 也会根据训练 Token 的总量动态调整 LR 的衰减终点, 以保持在海量数据下模型不会过早“停止学习”。

For chinchilla analysis: WSD-style learning rate

Deepseek uses WSD-style learning rate – fast warmup + two decay steps of 10% each

A multi-step learning rate scheduler is employed during pre-training instead of the typical cosine scheduler. Specifically, the learning rate of the model reaches its maximum value after 2000 warmup steps, and then decreases to 31.6% of the maximum value after processing 80% of the training tokens. It further reduces to 10% of the maximum value after 90% of the tokens. The gradient clipping during the training phase is set to 1.0.



(a) Multi-step v.s. cosine learning rate decay



(b) Different proportions of multi-step stages

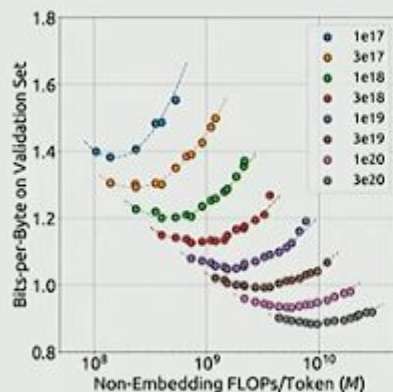
Generally seems to match performance of cosine learning rates.

Stanford

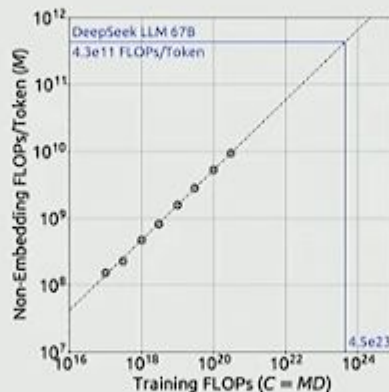
Use two step decay steps, allows chinchilla style analysis

Data-size tradeoff analysis: Chinchilla method 2

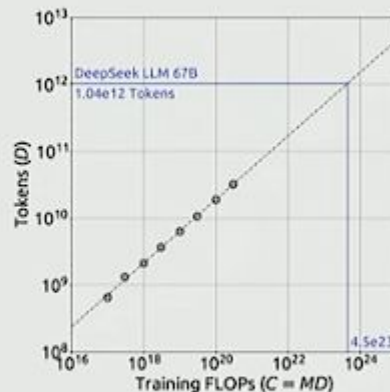
Straightforward isoflop-style analysis for selecting the model size tradeoffs.



(a) IsoFLOP curve



(b) Optimal model scaling



(c) Optimal data scaling

Stanford

Scaling predicts final model loss

The fitted scaling models (generally) accurately predict the final model losses.

Additionally, we fitted the loss scaling curve according to compute budget C and optimal generalization error, and predicted the generalization error for DeepSeek LLM 7B and 67B, as shown in Figure 5. The results indicate that using small-scale experiments can accurately predict

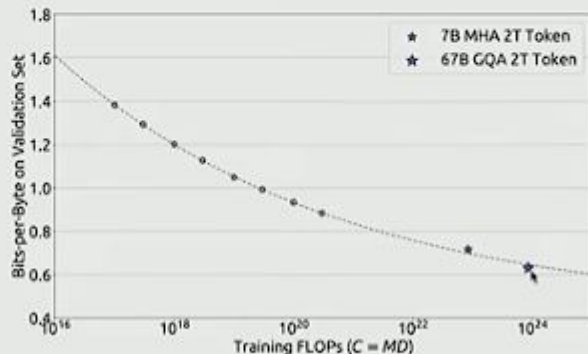


Figure 5 | Performance scaling curve. The metric is the bits-per-byte on the validation set. The dotted line represents the power law fitting the smaller model (grey circles). The blue stars represent DeepSeek LLM 7B and 67B. Their performance is well-predicted by the scaling curve.

LLaMA 3 (2024) Scaling laws

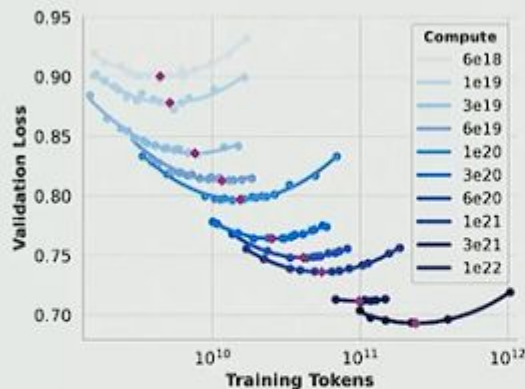
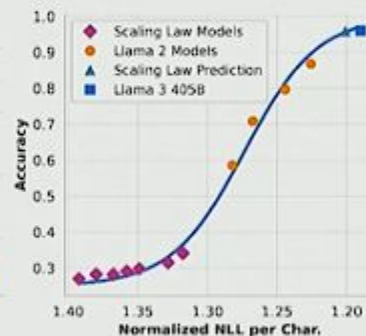
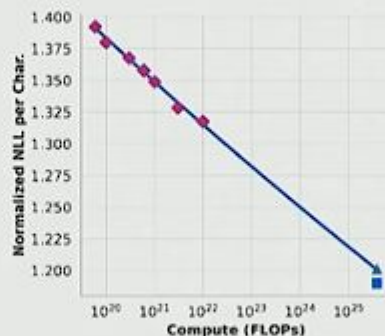


Figure 2 Scaling law IsoFLOPs curves between 6×10^{18} and 10^{22} FLOPs. The loss is the negative log-likelihood on a held-out validation set. We approximate measurements at each compute scale using a second degree polynomial.

Isoflops-style scaling (39-1 ratio)



Stanford
Compute-to-downstream scaling



Maybe due to better optimization to the model or data quality;
Need convert from compute to other metrics (sigmoid)

Hunyuan-1 (2024) large scaling laws

Yet more isoflops-style scaling (but this time for MoE parameter sizes)

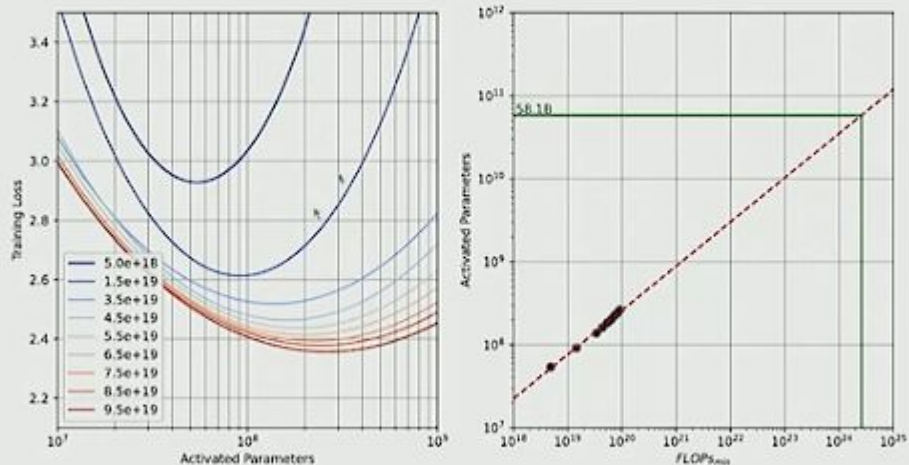


Figure 3: Using quadratic polynomial fitting, we obtain the scaling law of the optimal number of activation parameters under different minimum compute budgets.

Stanford

Optimal ratio – 96-1 (data to active param)

High ratio because of MOE (best replicate in 2025)

How much cost paying in terms of performance of model from using softmax attention to lightening and to hybrid-lightening

Architecture scaling laws + Chinchilla method 1

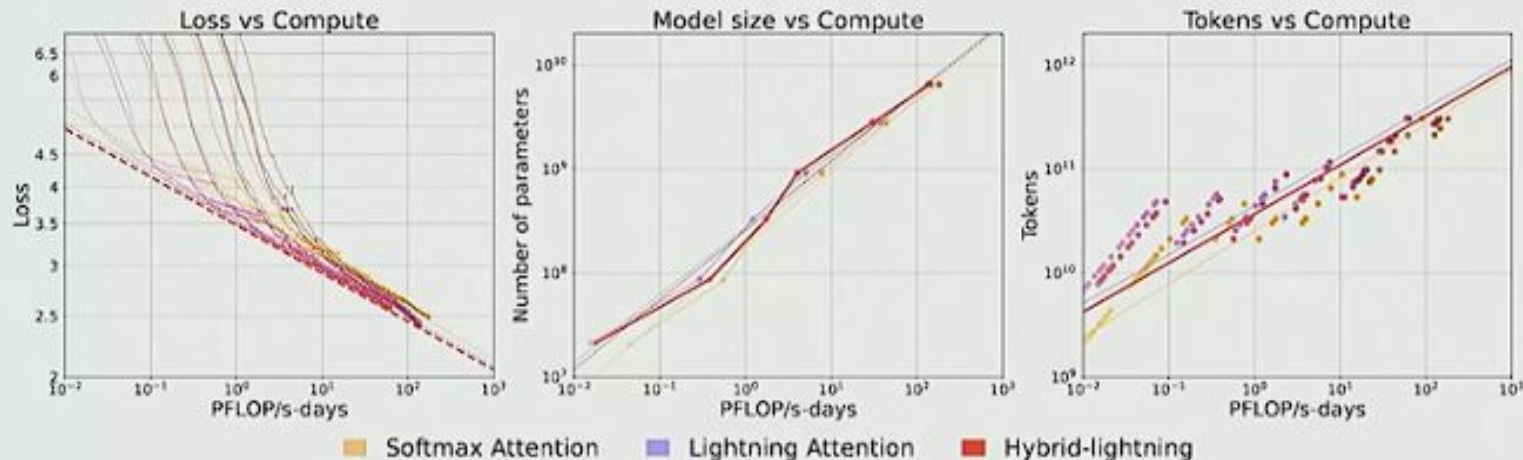


Figure 6 | **Summary of Scaling Laws.** Training curves (left) span models from 70M to 7B parameters. Optimal model size (center) and training tokens (right) are derived based on a specified compute budget estimation.

Based on the conclusion, they said its OK to train on long context models on basis of these architectures

Recent scaling law recipes

CerebrasGPT

- Use muP to make hyperparams invariant to scale
- Directly use the chinchilla scaling formula

DeepSeek recipe

- Assume most transformer hypers are invariant to scale
- Do a scaling analysis on batch / LR to figure out optimal scaling
- IsoFLOP analysis to figure out model sizing
 - › Use a piecewise-linear schedule to make chinchilla scaling cheap.

miniCPM recipe

- Use muP to make transformer + LR invariant to scale
- Use a piecewise linear schedule to get sample for Chinchilla method 3 (curve fitting)

Recent (late 2024+) but less detailed

LLaMA 3 / Hunyuan

- Just isoflops (no other scaling details)

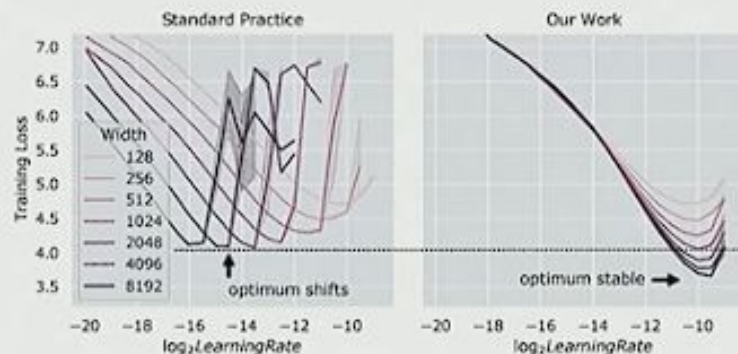
Minimax

- Architecture choice / decision scaling

Stanford

Validating and understanding μ P

“Scale invariant” hyperparameter tuning would be very useful



CerebrasGPT and miniCPT also use μ P – is it actually useful?

Preprint

A Large-Scale Exploration of μ -Transfer

Lucas Dax Lingle

Stanford



What is muP, anyway?

A Spectral Condition for Feature Learning

Greg Yang*
xAI

James B. Simon*
UC Berkeley & Imbue

Jeremy Bernstein*
MIT

(this is a very accessible 'muP for babies' paper)

muP is based off the following assertion. As a function of the width of the network n_l .

A1: The activations at initialization should remain $\Theta(1)$ Remain roughly constant, shouldn't blow up/vanish

A2: After one gradient step, the change in activation should be $\Theta(1)$

Note: if individual activations are $\Theta(1)$, then the norm should be $\Theta(\sqrt{n_l})$

Stanford



Deriving muP (condition A1)

h- activation

Simple architecture
assumption

Suppose that we have a simple, deep linear network ($h_l = W_l h_{l-1}$) and we init $W_l \sim N(0, \sigma I_{n_l \times n_{l-1}})$ then by basic matrix concentration $\|W_l\|_* \rightarrow \sigma(\sqrt{n_{l-1}} + \sqrt{n_l})$ and,

$$\|h_l\|_2 \approx \|W_l\|_* \|h_{l-1}\|_2$$

Now let's pick $\sigma = \frac{\sqrt{n_l}}{\sqrt{n_{l-1}}} (\sqrt{n_l} + \sqrt{n_{l-1}})^{-1} = \Theta\left(\frac{1}{\sqrt{n_{l-1}}} \min\left(1, \sqrt{\frac{n_l}{n_{l-1}}}\right)\right)$. What happens?

Inductive assumption- $\|h_{l-1}\|_2 = \Theta(\sqrt{n_{l-1}})$

Inductive case - $\|W_l\|_* \rightarrow \sigma(\sqrt{n_{l-1}} + \sqrt{n_l}) = \frac{\sqrt{n_l}}{\sqrt{n_{l-1}}}$

$$\|h_l\|_2 = \sqrt{n_l} + o(\sqrt{n_l})$$

Stanford



[Comments – the $\approx$ here is a bit hand-wavy, as the min s.v. of a Gaussian is $\sqrt{n_l} - \sqrt{n_{l-1} - 1}$, and is non-uniform, like J.L.]

Deriving muP (condition A2)

Now we need to deal with updates. Suppose we have the update ΔW_l on the weights. For SGD, on a linear layer, this looks like a rank-one loss-activation outer product.

$$\Delta W_l = -\eta_l \nabla_{h_l} \ell h_{l-1}^\top$$

Thus, $\|\Delta W_l h_{l-1}\|_2 = \|\Delta W_l\|_* \|h_{l-1}\|_2$. Now note that we have the update

$$\Delta h_l = W_l \Delta h_{l-1} + \Delta W_l (h_{l-1} + \Delta h_{l-1})$$

Assuming that the leading order terms don't cancel, we see that

- $W_l \Delta h_{l-1} = \Theta(\sqrt{n_l})$ from induction assumption + condition A1 argument
- $\Delta W_l h_{l-1} = \|\Delta W_l\|_* \sqrt{n_{l-1}}$ from above, thus $\|\Delta W_l\|_* = \Theta\left(\frac{\sqrt{n_l}}{\sqrt{n_{l-1}}}\right)$
- $\Delta W_l \Delta h_{l-1} = O(\|\Delta W_l\|_* \sqrt{n_{l-1}})$

Stanford



Deriving muP (condition A2) part 2

Recall – we want all 3 terms of $\Delta h_l = W_l \Delta h_{l-1} + \Delta W_l (h_{l-1} + \Delta h_{l-1})$ to be $\Theta(\sqrt{n_l})$
And the key is to pick LR such that $\|\Delta W_l\|_* \sqrt{n_{l-1}} = \Theta(\sqrt{n_l})$. How can we do that?

Suppose that the loss update *also* scales $O(1)$. Then we can write down..

$$\Delta \ell = \Theta(\langle \Delta W_l, \nabla_{W_l} \ell \rangle) = \Theta(\|\Delta W_l\|_F \|\nabla_{W_l} \ell\|_F) = \Theta(\|\Delta W_l\|_* \|\nabla_{W_l} \ell\|_*)$$

Where we use the fact that $\Delta W_l = -\eta \nabla_{W_l} \ell$ in standard SGD updates.

Now plug in $\Delta \ell = O(1)$, $\|\Delta W_l\|_* = \Theta\left(\frac{\sqrt{n_l}}{\sqrt{n_{l-1}}}\right)$ to get that $\|\nabla_{W_l} \ell\|_* = \Theta\left(\frac{\sqrt{n_{l-1}}}{\sqrt{n_l}}\right)$

Finally, from the previous slide, recall that $\Delta W_l = -\eta_l \nabla_{h_l} \ell h_{l-1}^T$ and thus

$$\eta_l = \Theta\left(\frac{n_l}{n_{l-1}}\right)$$

Stanford



[with Adam, $\|\Delta W_l\|_* \sqrt{n_{l-1}} = \Theta(1)$]

Adam is different 1/fan-in

muP mini recap..

So, what is (baby) muP about? Controlling activations (and changes) via W and ΔW

Initialization: Set to $\Theta\left(\frac{1}{\sqrt{n_{l-1}}} \min\left(1, \sqrt{\frac{n_l}{n_{l-1}}}\right)\right)$

Learning rates: Set to $\frac{n_l}{n_{l-1}}$ (for Adam $\frac{1}{n_{l-1}}$)

Compared to 'standard' parametrizations – these set

Initialization: Set to $\frac{1}{\sqrt{n_{l-1}}}$

Learning rates: Set to $\Theta(1)$

Differences – LR changes for Adam, also init diffs when fanout $n_l < \text{fanin}$

Stanford



Implementation in Cerebras GPT

We now have the background to follow cerebrasGPT

Variables	Standard Parameterization (SP)	Maximal Update (μP)
W	A multiplicative or fully-connected weights tensor	
b	A bias weights tensor	
X, Y	Activation tensors: layer input, output, respectively	
$d_{model, base}$	Proxy (base) model's layer width	
d_{model}	Width of each layer	
d_{head}	Size of each attention head	
$embed$	Combined tokens + position embedding function	
η_{base}	The base learning rate (LR): Maximum in training schedule	
σ_{base}	The base initialization standard deviation	
m_{width}	—	Layer width multiplier: $d_{model}/d_{model, base}$
m_{emb}	—	Embedding output multiplier

	Standard Parameterization (SP)	Maximal Update (μP)
Empirically Tuned Values		
$d_{model, base}$	—	256
η_{base}	Must tune for each model size	6e-3
σ_{base}	0.02	0.08
m_{emb}	—	10.0
Formulas		
Embedding initializer	$W_{emb} \sim N_{trunc}(0, \sigma_{base}^2)$	
Embedding LR	$\eta_{emb} = \eta_{base}$	
Embedding output	$Y_{emb} = embed(X)$	$Y_{emb} = m_{emb} \cdot embed(X)$
LN initializer	$W_{\gamma} \sim 1, b_{\beta} \sim 0$	
LN LR	$\eta_{LN} = \eta_{base}$	
Bias initializer	$b \sim 0$	
Bias LR	$\eta_b = \eta_{base}$	
MHA equation	$\text{softmax}\left(\frac{Q^T K}{\sqrt{d_{head}}}\right) V$	$\text{softmax}\left(\frac{Q^T K}{d_{head}}\right) V$
QKV weights initializer	$W_{qkv} \sim N_{trunc}(0, \sigma_{base}^2)$	$W_{qkv} \sim N_{trunc}(0, \sigma_{base}^2/m_{width})$
QKV weights LR	$\eta_{qkv} = \eta_{base}$	$\eta_{qkv} = \eta_{base}/m_{width}$
O weights initializer	$W_o \sim N_{trunc}(0, \frac{\sigma_{base}^2}{2 m_{width}})$	$W_o \sim N_{trunc}(0, \frac{\sigma_{base}^2}{2 m_{width} m_{width}})$
O weights LR	$\eta_o = \eta_{base}$	$\eta_o = \eta_{base}/m_{width}$
fn1 weights initializer	$W_{fn1} \sim N_{trunc}(0, \sigma_{base}^2)$	$W_{fn1} \sim N_{trunc}(0, \sigma_{base}^2/m_{width})$
fn1 weights LR	$\eta_{fn1} = \eta_{base}$	$\eta_{fn1} = \eta_{base}/m_{width}$
fn2 weights initializer	$W_{fn2} \sim N_{trunc}(0, \frac{\sigma_{base}^2}{2 m_{width}})$	$W_{fn2} \sim N_{trunc}(0, \frac{\sigma_{base}^2}{2 m_{width} m_{width}})$
fn2 weights LR	$\eta_{fn2} = \eta_{base}$	$\eta_{fn2} = \eta_{base}/m_{width}$
Output logits multiplier	$Y_{logits} = W_{out} emb(X)$	$Y_{logits} = \frac{W_{out} emb(X)}{m_{width}}$

Stanford



Embedding doesn't scale
Initialization + LR

Deeper dive into μ P

Recall – μ P is a scaling procedure for hyperparams (as a function of width)

Param	Init Variance (Θ)	Adam LR (Θ)	Init Variance (Exact)	Adam LR (Exact)
$\mathbf{W}^E$	1	1	1	α
$\mathbf{W}^{AQ}$	$1/M$	$1/M$	$1/M$	$\alpha P/M$
$\mathbf{W}^{AK}$	$1/M$	$1/M$	$1/M$	$\alpha P/M$
$\mathbf{W}^{AV}$	$1/M$	$1/M$	$1/M$	$\alpha P/M$
$\mathbf{W}^{AO}$	$1/(HD)$	$1/(HD)$	$1/M$	$\alpha P/M$
$\mathbf{W}^{FI}$	$1/M$	$1/M$	$1/M$	$\alpha P/M$
$\mathbf{W}^{FO}$	$1/F$	$1/F$	$0.25/M$	$\alpha P/M$
$\mathbf{W}^U$	$1/M^2$	$1/M$	$1/M^2$	$\alpha P/M$

Embedding

Attention params

Input/output MLP MM

Softmax linear

Table 2: μ P scaling rules for transformers; a rule for attention scale is detailed in text below.

In addition, μ P uses an attention scale of $\tau^{-1} = \Theta(1/D)$ instead of the usual $\tau^{-1} = 1/\sqrt{D}$. For simplicity, we use $\tau^{-1} = 1/D$, since in preliminary experiments we observed only a small improvement from using smaller multiples of $1/D$. Note that for D fixed across model widths M , any constant $\tau^{-1} \neq 0$ technically complies with μ P (Yang et al., 2021) but in the experiments, τ^{-1} will be shown to have a major impact on performance and transfer.

Stanford



在传统的 Transformer(如 GPT-2 或早期 Llama)中, 我们习惯使用 $1/\sqrt{d}$ (即维度 d 的平方根倒数)来缩放 Attention 或初始化, 这被称为 **Standard Parameterization (SP)**。

而 **μ P (Maximal Update Parameterization)** 提倡在某些层(尤其是 Output Head 和部分 Attention 缩放)使用 $1/d$ 。这背后的核心逻辑是为了实现**决策不变性 (Alignment)**, 即: 无论模型变多大, 其特征激活值的幅度和梯度的更新尺度保持一致。

为什么 μ P 觉得不够? 在模型训练过程中, 神经元不再是独立的随机变量, 它们会产生高度的相关性 (Feature

Scaling protocol in the work

Architecture – mostly similar to what's in the class

Important limitation of the work: *only width scaling*

We use the following default configuration, deviating from it only if specifically mentioned. The depth is fixed at $L = 24$, and we consider model widths $M \in \{128, 512, 2048\}$, yielding three model sizes ranging from 4.7M to 1.2B non-embedding parameters. The head width is fixed at $D = 128$, the number of heads is $H = M/D$, and MLP hidden width is $F = 4M$. The models use RMS LayerNorm without gains (Zhang & Sennrich, 2019), linear projections without biases (Raffel et al., 2020), RoPE on the queries and keys (Su et al., 2021), and ReLU for the MLP nonlinearity (Vaswani et al., 2017; Raffel et al., 2020).

Transformer

Want Optimal LR stable

Replicating μ P

Q1: Does μ P work as claimed? When we scale widths, is optimal LR constant?

Ablation	Width	Base LR					Transfer
		2^{-10}	2^{-8}	2^{-6}	2^{-4}	2^{-2}	
Baseline μ P	128	3.846	3.743	3.695	3.884	4.143	✓
	512	3.114	2.993	2.953	3.221	3.506	
	2048	2.711	2.553	2.511	2.563	3.244	
Projection Biases	128	3.838	3.735	3.705	3.911	4.269	✓
	512	3.108	2.986	2.947	2.970	3.557	
	2048	2.710	2.552	2.529	2.672	3.418	

As shown in Table 1, the learning rates transfer reliably across model sizes under μ P. Despite each model being 4x wider (and 16x larger) than the last, the smallest model's optimal base learning rate α directly predicts the optimum in our sweeps for the larger models.

Stanford



It does work.transfer reliably

What is muP robust to?

Modern LMs have many components that deviate from muP's theory

- Activations – SwiGLU and squared relu
- Batch sizes – Large / small
- Initialization variations – zero attention, etc.
- RMS norm gains
- Exotic optimizers (Lion)
- Regularizers

Which of these (if any) break muP?

What is muP robust to? Nonlinearities

SwiGLU, Squared ReLU have the same optimal LR (and both provide minor gains)

Ablation	Width	Base LR					Transfer
		2^{-10}	2^{-8}	2^{-6}	2^{-4}	2^{-2}	
Baseline μ P	128	3.846	3.743	3.695	3.884	4.143	✓
	512	3.114	2.993	2.953	3.221	3.506	
	2048	2.711	2.553	2.511	2.563	3.244	
SwiGLU Nonlinearity	128	3.800	3.740	3.715	4.090	7.024	✓
	512	3.070	2.975	2.953	3.175	6.863	
	2048	2.677	2.536	2.505	2.553	4.571	
Squared ReLU Nonlinearity	128	3.808	3.735	3.686	3.999	4.484	✓
	512	3.071	2.964	2.929	3.184	7.299	
	2048	2.666	2.516	2.482	2.532	3.259	

What is muP robust to? Batch size

Larger and smaller batches.

The original derivation doesn't handle batch size considerations.

Ablation	Width	Base LR					Transfer
		2^{-10}	2^{-8}	2^{-6}	2^{-4}	2^{-2}	
Baseline μ P	128	3.846	3.743	3.695	3.884	4.143	✓
	512	3.114	2.993	2.953	3.221	3.506	
	2048	2.711	2.553	2.511	2.563	3.244	
4x Larger Batch	128	3.844	3.735	3.697	3.716	10.380	✓
	512	3.141	2.990	2.965	3.305	10.373	
	2048	2.745	2.556	2.541	2.697	7.197	
4x Smaller Batch	128	3.855	3.774	3.736	3.945	4.104	✓
	512	3.120	3.011	2.977	3.024	3.521	
	2048	2.714	2.568	2.527	2.549	3.223	

What is muP robust to? initialization

There are new initializations that are sometimes used

- SP Unembedding – This is the pre-softmax linear layer. $1/M$ (SP) vs $1/M^2$ (muP)
- Zero Query – Set the query matrix to zero (so that all items get uniform attention)

SP Unembedding Init	128	3.861	3.765	3.699	3.896	4.161	✓
	512	3.119	2.990	2.951	3.265	3.582	
	2048	2.716	2.554	2.509	2.564	7.471	
Zero Query Init	128	3.836	3.743	3.694	3.877	4.167	✓
	512	3.115	2.992	2.949	3.135	3.532	
	2048	2.711	2.553	2.510	2.551	3.272	

What is muP not robust to? RMSnorm gain

In our arch – RMSNorm has learnable gains. This turns out to break muP

Ablation	Width	Base LR					Transfer
		2 ⁻¹⁰	2 ⁻⁸	2 ⁻⁶	2 ⁻⁴	2 ⁻²	
Baseline μ P	128	3.846	3.743	3.695	3.884	4.143	✓
	512	3.114	2.993	2.953	3.221	3.506	
	2048	2.711	2.553	2.511	2.563	3.244	
RMSNorm Gains (Vector)	128	3.842	3.744	3.689	3.670	3.681	✗
	512	3.101	2.992	2.951	2.950	3.412	
	2048	2.692	2.553	2.609	2.605	3.169	
RMSNorm Gains (Scalar)	128	3.843	3.749	3.692	3.670	4.471	✗
	512	3.106	3.000	2.961	2.959	3.515	
	2048	2.704	2.570	2.525	2.542	3.334	

As shown in Table 1, optimal learning rates for these models *do not* reliably transfer when using $\Theta(1)$ learning rate scaling for the gains, despite the fact that the ‘coordinate size’ of the features before and after RMS Normalization is $\Theta(1)$ w.r.t. width by design. In addition to the lack of transfer in these experiments, we find trainable gains harm the quality of the largest μ P models when the base learning rate α is optimal. In Section 4.5, we also find that standard transformers with trainable gains underperform μ P transformers without them.

Stanford

But these gains can be removed with little loss of perf..

Adding learnable gains breaks muP

What is muP not robust to? Exotic optimizers

There are other, exotic optimizers based on just gradient signs. Do they transfer?

Algorithm 1 AdamW Optimizer

```

given  $\beta_1, \beta_2, \epsilon, \lambda, \eta, f$ 
initialize  $\theta_0, m_0 \leftarrow 0, v_0 \leftarrow 0, t \leftarrow 0$ 
while  $\theta_t$  not converged do
   $t \leftarrow t + 1$ 
   $g_t \leftarrow \nabla_{\theta} f(\theta_{t-1})$ 
  update EMA of  $g_t$  and  $g_t^2$ 
   $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
   $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
  bias correction
   $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
   $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
  update model parameters
   $\theta_t \leftarrow \theta_{t-1} - \eta_t (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta_{t-1})$ 
end while
return  $\theta_t$ 

```

Algorithm 2 Lion Optimizer (ours)

```

given  $\beta_1, \beta_2, \lambda, \eta, f$ 
initialize  $\theta_0, m_0 \leftarrow 0$ 
while  $\theta_t$  not converged do
   $g_t \leftarrow \nabla_{\theta} f(\theta_{t-1})$ 
  update model parameters
   $c_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
   $\theta_t \leftarrow \theta_{t-1} - \eta_t (\text{sign}(c_t) + \lambda \theta_{t-1})$ 
  update EMA of  $g_t$ 
   $m_t \leftarrow \beta_2 m_{t-1} + (1 - \beta_2) g_t$ 
end while
return  $\theta_t$ 

```

Lion Optimizer	128	3.708	3.736	4.057	4.344	10.380
	512	2.952	2.947	3.416	3.961	10.285
	2048	2.519	2.511	3.151	10.377	10.377

Stanford

What is muP not robust to? – (strong) weight decay

What about strong (0.1) weight decay? – this is maybe the only significant muP failure

Decoupled Weight Decay	128	3.760	3.679	3.694	3.741	4.011
	512	3.057	2.963	2.957	3.139	3.373
	2048	2.686	2.535	2.502	3.123	6.594

Algorithm 1 SGD with L₂ regularization and SGD with decoupled weight decay (SGDW), both with momentum

```

1: given initial learning rate  $\alpha \in \mathbb{R}$ , momentum factor  $\beta_1 \in \mathbb{R}$ , weight decay/L2 regularization factor  $\lambda \in \mathbb{R}$ 
2: initialize time step  $t \leftarrow 0$ , parameter vector  $\theta_{t=0} \in \mathbb{R}^n$ , first moment vector  $m_{t=0} \leftarrow \theta$ , schedule multiplier  $\eta_{t=0} \in \mathbb{R}$ 
3: repeat
4:    $t \leftarrow t + 1$ 
5:    $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$  ▷ select batch and return the corresponding gradient
6:    $g_t \leftarrow \nabla f_t(\theta_{t-1}) \odot \lambda \theta_{t-1}$ 
7:    $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$  ▷ can be fixed, decay, be used for warm restarts
8:    $m_t \leftarrow \beta_1 m_{t-1} + \eta_t g_t$ 
9:    $\theta_t \leftarrow \theta_{t-1} - m_t$  ▷  $\eta_t \lambda \theta_{t-1}$ 
10: until stopping criterion is met
11: return optimized parameters  $\theta_t$ 

```

Stanford



Is muP useful? At least to some extent..

Overall, muP generally seems useful – insofar that SP is quite a bit more unstable.

Width	LR				
	2^{-10}	2^{-8}	2^{-6}	2^{-4}	2^{-2}
128	3.841	3.757	3.706	3.879	4.030
512	3.013	2.967	2.987	3.383	7.403
2048	2.738	2.902	7.247	7.477	7.314

Table 3: Validation losses for SP models.

Params	Width	Base LR		
		2^{-8}	2^{-6}	2^{-4}
2M	128	3.791	3.766	3.814
40M	512	3.016	2.983	3.004
600M	2048	2.513	2.459	2.466
10B	8192	2.238	2.167	2.169

Table 4: Validation losses for our large-scale experiment.

Current evidence suggests that muP parametrization / initialization may be easier to tune.

Stanford



Recap: scaling in the wild

What are challenges in scaling 'in practice'

1. Setting model arch hyperparameters (width, etc)
2. Setting optimizer hyperparameters (LR, batch)
3. Compute needed to fit the big chinchilla sweep

Some solutions?

1. Assume stability (or use muP)
2. Search for optimal LR / batch in small scale, either keep fixed or predict scaling
3. Use alternative learning schedules (WSD-like)